

ORION - TELJES RENDSZERDOKUMENTÁCIÓ

Fejlesztők



Perczel Péter Géza



Stieber Domonkos

Ez a dokumentum az Orion rendszer teljes és hivatalos dokumentációja. Bemutatja a projekt célját, a rendszer működését, a frontend és backend architektúrát, az adatmodellt, az integrációkat, valamint a fejlesztési és üzemeltetési szempontokat.

Tartalomjegyzék

1. Bevezetés

- 1.1. Projekt célja
- 1.2. Probléma bemutatása és az Orion megoldása

2. Projektkép és üzleti kontextus

- 2.1. Kik használják a rendszert?
- 2.2. Mi az üzleti érték röviden?
- 2.3. A rendszer fő területei

3. Rendszerszintű architektúra

4. Technológiai verem és tervezési döntések

- 4.1. Frontend technológiai döntések
 - 4.1.1. Miért React és Vite?
 - 4.1.2. Miért TypeScript?
 - 4.1.3. Állapotkezelés
 - 4.1.4. Routing és adatlekérés
 - 4.1.5. Megjelenés és Stílusok
 - 4.1.6. Technológiai Verem Összefoglaló
- 4.2. Backend Technológiai Döntések
 - 4.2.1. Miért Node.js és Express 5?
 - 4.2.2. Biztonság, Validáció és Hitelesítés
 - 4.2.3. Adatbázis és Realtime Képességek
 - 4.2.4. Fájlkézelés és Értesítések
 - 4.2.5. Backend Technológiai Verem Összefoglaló
- 4.3. A választott kombináció előnyei

5. Adatmodell és adatbázislogika

- 5.1. Adatbázis és Technológiai Háttér
- 5.2. Adatbázis sémakép
- 5.3. Táblák részletes leírása
 - 5.3.1. Felhasználói (Diák) Modul
 - 5.3.2. Vállalati (Cég) Modul
 - 5.3.3. Hirdetések és Munkaviszony (ATS Modul)
 - 5.3.4. Kommunikáció és Értesítési Rendszer
- 5.4. Referenciális Integritás
- 5.5. Szerveroldali Tárolt Eljárások (RPC)
- 5.6. Eseményvezérelt Adatbázis Logika (Triggerek)

6. Backend Szerver Architektúra

- 6.1. Hálózati Middleware és Realtime Kommunikáció
- 6.2. API Végpontok és Szolgáltatások
 - 6.2.1. Autentikációs Modul
 - 6.2.2. Felhasználói (Diák) Modul
 - 6.2.3. Cég Modul
 - 6.2.4. Álláshirdetések
 - 6.2.5. Jelentkezések
 - 6.2.6. Kommunikáció és Chat
 - 6.2.7. Rendszerüzenetek
 - 6.2.8. Email Hitelesítés és Jelszó
 - 6.2.9. OrionAI Mikroszolgáltatás
- 6.3. Hibakezelési Stratégia

7. Frontend architektúra

- 7.1. Könyvtárstruktúra
- 7.2. Komponensek és felelősségek
- 7.3. Hozzáférésvédelem
- 7.4. Az oldal részei és funkciói
 - 7.4.1. Feature modulok áttekintése
 - 7.4.2. Publikus Modul
 - 7.4.3. Globális Autentikációs Modul
 - 7.4.4. Munkaadói (Céges) Modul
 - 7.4.5. Álláskeresői (Diák) Modul
 - 7.4.6. Kommunikációs Modul
- 7.5. Adatkezelés
- 7.6. Űrlapok és validáció
- 7.7. Többnyelvűség
- 7.8. SEO és Statikus Eszközök
- 7.9. Reszponzivitás

8. Külső Integrációk és Speciális Szolgáltatások

- 8.1. Email Integráció és Automatizáció
- 8.2. OrionAI Integráció (Python NLP)

9. Biztonság, jogosultság és adatvédelem

- 9.1. Auth és session biztonság
- 9.2. Hibák kezelése és visszajelzés
- 9.3. Szerepköralapú hozzáférés
- 9.4. Titkosított dokumentumkezelés
- 9.5. Adatminimalizálás és elkülönítés
- 9.6. Frontend oldali biztonsági megfontolások
- 9.7. Kulcsok és környezeti változók

10. Minőségbiztosítás, tesztelés és teljesítmény

- 10.1. Tesztelési eszközök és stratégia
- 10.2. Manuális API Validáció és Backend Integráció
- 10.3. Teljesítmény és skálázási megfontolások
- 10.4. Lighthouse Audit és mérőszámok

11. Fejlesztői működés, build és üzemeltetés

- 11.1. Helyi fejlesztői környezet
- 11.2. Környezeti változók
- 11.3. Build és release szemlélet

12. Jövőbeli fejlesztési irányok és tervek

- 12.1. Üzleti és monetizációs tervek
- 12.2. Felhasználói élmény és kényelmi funkciók
- 12.3. Technikai és infrastrukturális továbbfejlesztések

13. Fejlesztési folyamat és ütemezés

- 13.1. Csapat és szerepkörök
- 13.2. Eszközök és kommunikáció
- 13.3. Fejlesztési ütemterv
- 13.4. Fejlesztési folyamat összefoglalása

14. Mellékletek

1. Bevezetés

TL;DR: Az Orion egy **közvetlen diákmunka-platform**, amely a bürokrácia drasztikus csökkentését és a diákok számára elérhető bérek maximalizálását célozza meg. A rendszer kiküszöböli a közvetítőket, így a cégek és a diákok közvetlenül kommunikálhatnak és szerződhetnek, ami mindkét fél számára anyagilag előnyösebb és átláthatóbb folyamatot eredményez.

1.1. Projekt célja

Az Orion egy modern diákmunka megosztó és közvetítő platform, hasonlóan a hazai piacon ismert profession.hu-hoz, azonban kifejezetten a diákmunkára és a diákok számára optimalizált változatban.

A projekt legfőbb küldetése, hogy egy transzparens, gyors és közvetlen kapcsolatot biztosítson a munkát kereső diákok és az őket foglalkoztatni kívánó cégek között, kihagyva a hagyományos, lassú és gyakran a diákok számára anyagilag hátrányos harmadik feleket, például a közvetítő diákszövetkezeteket.

1.2. Probléma bemutatása és az Orion megoldása

A jelenlegi diákmunka-piacon a diákok legtöbbször diákszövetkezeteken keresztül helyezkednek el és dolgoznak. Ez a rendszer gyakran kényelmetlen, adminisztrációval és bürokráciával terhelt a bonyolult szerződéskötési folyamatok miatt. Továbbá a közvetítők beiktatása miatt a diákok sok esetben jelentősen alacsonyabb órabért kapnak kézhez ahhoz képest, amit a cégek ténylegesen kifizetnek értük.

A megoldás: Az Orion rendszerében a diákok és a munkáltató cégek *közvetlenül* egymással szerződnek és tartják a kapcsolatot. A platform piacteret, biztonságos kommunikációt és adminisztrációs támogatást biztosít számukra. Ezzel jelentősen csökken a szerződéskötési és adminisztrációs teher, a diákok pedig nem kerülnek anyagi hátrányba a közvetítői modell miatt. A cégek modern önkiszolgáló felületen oszthatják meg az álláshirdetéseiket, közvetlenül érhetik el a jelentkező diákokat, és maguk dönthetnek a kiválasztási folyamatról.

2. Projektkép és üzleti kontextus (Business Context)

TL;DR: A rendszer a diákok és cégek közötti **transzparens piacot** teremti meg, ahol profilkezelés, jelentkezéskövetés és valós idejű chat támogatja a hatékony együttműködést. Célja, hogy egy modern, adatvezérelt digitális munkateret biztosítson, amely gyorsítja a toborzást és csökkenti az adminisztrációs terheket mindkét oldal számára.

2.1. Kik használják a rendszert? (Primary Actors)

A rendszer elsődleges szereplői

Szereplő	Fő cél	Főbb funkciók
Diák / álláskereső	Munkalehetőség keresése és kezelése	Regisztráció, profilkitöltés, önéletrajz feltöltés, jelentkezés, chat, státuszkövetés
Cég / munkaadó	Jelöltek elérése és munkák meghirdetése	Regisztráció, cégprofil kezelése, álláshirdetés létrehozása, ATS, kommunikáció, dashboard

2.2. Mi az üzleti érték röviden? (Business Value)

Az Orion értéke három különböző nézőpontból ragadható meg:

- Diákok számára:** Egy koncentrált, modern és kevés adminisztrációval járó felületet ad.
- Cégek számára:** Gyorsabb jelöltelérést, önkiszolgáló hirdetéskezelést és egyetlen rendszerbe integrált kommunikációt biztosít.
- Platform szinten:** Egy olyan adatvezérelt működést (Data-driven Operation) tesz lehetővé, amelyben a jelentkezések, profilmegtekintések, chat-partnerek, dokumentumok és dashboard-statisztikák egymásra épülnek.

2.3. A rendszer fő területei (Core Domains)

- **Felhasználói rész (User Domain):** diák fiókok, hitelesítés, profil, dokumentumok, statisztikák.
- **Céges rész (Company Domain):** cégfiókok, hitelesítés, cégprofil, dashboard, jelöltkezelés.
- **Hirdetési rész (Advertisement Domain):** álláshirdetések létrehozása, listázása, frissítése, keresése és láthatósága.
- **Jelentkezési rész (Application Domain):** jelentkezések, státuszok, ATS (Applicant Tracking System) folyamatok, felvétel vagy elutasítás.
- **Kommunikációs rész (Communication Domain):** chat, rendszerüzenetek, olvasási állapotok, valós idejű (Realtime) frissítések.
- **Biztonsági rész (Security Domain):** auth, tokenek, titkosított személyes adatok, munkamenet (Session) és útvonalvédelem (Route Protection).
- **Intelligens szolgáltatások (AI Services):** OrionAI és az ehhez kapcsolódó külön útvonalak.

3. Rendszerszintű architektúra

TL;DR: A projekt modern, **rétegzett architektúrát** követ, ahol a React alapú kliens, a Node/Express szerver és a Supabase adatbázis alkot technológiailag zárt egységet. Ez a felépítés biztosítja a magas teljesítményt, a valós idejű adatcserét, valamint a könnyű skálázhatóságot és karbantarthatóságot a moduláris domén-alapú tervezés révén.

3.1. Magas szintű architektúra

Az Orion klasszikus webalkalmazás-architektúrát követ, de több olyan elemmel egészül ki, amelyek a termék valós idejű és adatintenzív működéséhez szükségesek. A felhasználók böngészőből érik el a React alapú klienst. A kliens REST-szerű HTTP kérésekkel kommunikál az Express szerverrel, miközben bizonyos kommunikációs vagy eseményalapú funkciók realtime csatornán is elérhetők. Az adatok tartós tárolásáért és kezeléséért egy Supabase / PostgreSQL környezet felel, ahol a rendszer jelentős mértékben támaszkodik tárolt eljárásokra és adatbázis-szintű üzleti logikára. Az alábbiakban lehet látni a Architektúra folyamatábráját:



3.2. Rétegek és felelősségek

Réteg	Felelősség	Tipikus elemek
Prezentációs réteg	Felhasználói élmény, űrlapok, navigáció, megjelenítés	src/features, komponensek, route-ok, kliensoldali validáció
Alkalmazásréteg	Kérésfeldolgozás, auth, modulhatárok, API routing	Express route-ok, middleware, cookie és CORS kezelés
Szolgáltatási réteg	Üzleti logika, adatlekérés, állapotváltások	*Service.ts fájlok, domain-specifikus folyamatok
Adatréteg	Tartós tárolás, relációk, integritás, titkosítás	PostgreSQL táblák, FK kapcsolatok, RPC, triggerek
Realtime réteg	Valós idejű kommunikáció és események	RealtimeServer.ts, chathez kapcsolódó események

3.3. Moduláris szervezőelv

A projekt egyik erőssége, hogy a frontend és a backend is moduláris szervezést követ. A kliensoldalon a fő üzleti területekhez külön feature mappák tartoznak: public, auth, company, user, chat. A szerveroldalon ehhez hasonló felosztás jelenik meg a route és service párokban: auth, user, company, advertisement, applicant, message, systemmessage, email, OrionAI.

Ez a struktúra lehetővé teszi, hogy az Oriont ne technológiai, hanem üzleti modulok mentén lehessen fejleszteni. Egy új funkció bevezetése így általában végigvezethető ugyanazon a részen: frontend feature, API útvonal, service logika, adatbázis módosítás, tesztelés.

4. Technológiai verem és tervezési döntések

TL;DR: A **React 19**, **Vite** és **TypeScript** hármasa biztosítja az extrém gyors fejlesztési ciklust és a hosszú távon is fenntartható, típusbiztos kódázist. A backend oldalon az Express és a Supabase robusztus API-t és valós idejű képességeket nyújt, míg a szigorú adatvalidáció garantálja a rendszer integritását és biztonságát.

4.1. Frontend Technológiai Döntések (Detailed)

A modern webes alkalmazások alapjait nem egyszerű kiválasztani. Mérlegelnünk kellett a fejlesztési sebességet, a későbbi karbantarthatóságot és azt, hogy mennyire lesz gyors az oldal (performance). Az Orion frontendjénél az volt a cél, hogy egy stabil, jól bővíthető és szép rendszert építsünk. Ezért választottuk a React 19-et és a Vite-ot.

4.1.1. Miért pont React és Vite?

Sok más lehetőség is lett volna (például Angular vagy Vue), de ezek túl kötöttek lettek volna az Orionhoz. A React azért jó, mert az összetett felületeket (például az álláskeresőt vagy a jelöltkezelőt) könnyen kisebb, újrahasznosítható egységekre bonthatjuk (decomposition). A React gyorsasága és a Virtual DOM használata biztosítja, hogy a rengeteg adat megjelenítésekor se lassuljon be az oldal.

Régebben a legtöbb React projekt Webpacket használt, de az egyre lassabb (suboptimal) lett, ahogy nőtt a kód. A Vite viszont egy teljesen új megközelítést (paradigm shift) hozott. Nagyon gyorsan indul el, és a kód módosítása után is azonnal látszik a változás (Hot Module Replacement – HMR). Ez segít abban, hogy a fejlesztés ne lassuljon le akkor sem, ha már óriásira nőtt a projekt.

4.1.2. Miért jó nekünk a TypeScript?

Az egyik legfontosabb döntésünk az volt, hogy sima JavaScript helyett TypeScriptet használunk. Egy ekkora, megközelítőleg 25 000 soros alkalmazásnál sima JS-ben szinte lehetetlen lenne átlátni mindent, és sokkal több váratlan hiba csúszna be (kiszámíthatóság – predictability). A TypeScript használata alapvető váltást hozott: segít a kód szerkezetének pontos leírásában és a hibák korai megtalálásában.

TypeScript nélkül a kód módosítása (refactoring) nagyon kockázatos lenne, mert egy kis változtatás teljesen máshol is elronthatna dolgokat (side effects). A TypeScript már írás közben segít: figyelmeztet, ha valami nincs összhangban (coherence), például ha egy álláshirdetés adatainál olyasmit akarunk használni, ami nincs is ott (compatibility). Így kevesebb lesz a hiba, és a fejlesztőknek is könnyebb követni, hogy mi hogyan épül fel, ami biztosítja az Orion projekt hosszú távú stabilitását.

4.1.3. Állapotkezelés (State Management)

Az állapotkezelésnél azt akartuk, hogy a rendszer átlátható maradjon, és ne bonyolítsuk feleslegesen újabb könyvtárakkal. Sokan használnak Reduxot vagy MobX-et, de ezekhez rengeteg plusz kódot (boilerplate) kell írni, ami csak lassítja a munkát (development overhead).

Az Orionban a React saját megoldásait (Context API és Hooks) használjuk. A globális adatokat – például a bejelentkezést vagy az Üzenetküldőt (**OrionMessenger**) – külön részekre bontva kezeljük (pl. MessengerProvider). Minden más adat ott marad a helyén, ahol ténylegesen keletkezik, így a rendszer nem válik nehézkessé a folyamatos központi üzenetküldéstől. Mivel egy állásportál főként adatok lekéréséről szól, ez a megoldás bőven elég, és segít tisztán tartani a kódot.

4.1.4. Útvonalválasztás és Hálózat (Routing & Data Fetching)

Az oldalak közötti mozgást (routing) a react-router-dom 7-es verziója kezeli. Ez nem csak a navigációért felel, hanem a biztonságért is. Mivel az Orionban különböző szerepkörök vannak (keresők, cégek, vendégek), fontos, hogy mindenki csak azt lássa, amihez van joga. Ezt külön védelmi vonalakkal (Route Guards) oldottuk meg: ezek észrevétlenül (transparently) ellenőrzik a jogosultságot, mielőtt betöltenének egy-egy oldalt, így például egy sima álláskereső nem juthat be a cégek admin felületére.

Az adatok lekéréséhez (data fetching) saját megoldást használunk (native fetch), ami jól elkülönített és átlátható modulokra van bontva. Bár vannak már készen kapható komplex megoldások (például React Query), a mi rendszerünk jelenleg pontosan azt a gyorsaságot és átláthatóságot biztosítja, amire szükségünk van, anélkül, hogy külső csomagoktól függnénk. (nálunk ez a fájl az **IsLoggedIn.tsx**)

4.1.5. Megjelenés és Stílusok (Styling & UI)

A kinézetnél fontos döntés volt, hogy ne használjunk kész könyvtárakat (mint a Tailwind CSS vagy Material UI). Ezek bár gyorsak az elején, később korlátozhatják a tervezést, és feleslegesen nagyra növelik a letöltendő fájlokat (bundle bloat). Emellett egy 25 000 soros kódnál a Tailwind kódja nehezen olvashatóvá (readability) tette volna a fájlokat.

Helyette a sima CSS modern lehetőségeit (native CSS) használjuk, amit a Framer Motion animációival egészítettünk ki. Ez teljes szabadságot ad nekünk, hogy pontosan olyan szép és modern felületeket készítsünk, amilyeneket szeretnénk – legyen szó átlátszó üveg-hatású (glassmorphism) elemekről vagy a jelöltkezelő rendszer kártyáinak folyamatos animációjáról.

4.1.6. Technológiai Verem Összefoglaló

Kategória	Technológia	Leírás / Szerepkör
Mag (Core)	React 19	Komponens-alapú UI könyvtár és deklaratív renderelés.
Build Tool	Vite	Native ESM-alapú építőeszköz az extrém gyors fejlesztési ciklusokért.
Nyelv	TypeScript	Statikus típusbiztonság, IntelliSense és kódstabilitás.
Stílusozás	Vanilla CSS	Native CSS Variables, Flexbox és Grid a maximális kontrollért.
Állapotkezelés	Context API	Beépített, skálázható állapotkezelés boilerplate nélkül.
Navigáció	React Router 7	Kliensoldali útvonalválasztás és Route Guard rendszer.
Animációk	Framer Motion	Deklaratív, nagy teljesítményű UI animációk és átmenetek.
Űrlapok	React Hook Form	Optimális teljesítményű űrlapkezelés és validáció integráció.
Validáció	Zod	Séma-alapú típusbiztonság és adatvalidáció.
Ikonkészlet	Lucide React	Modern, könnyűsúlyú SVG ikonrendszer.
Visszajelzés	React Hot Toast	Reszponzív, nem blokkoló értesítési rendszer.

4.2. Backend Technológiai Döntések (Detailed)

TL;DR: Miért az **Express 5** és **Node.js**? (Könnyűsúlyú, aszinkron, TypeScripttel kiválóan tipizálható környezet). A valós idejű funkciókat és az adatbázist a **Supabase (PostgreSQL)** biztosítja, ami levéltárazza és titkosítja az adatokat. A rendszer egyedi **JWT** alapú hitelesítést és szigorú **Zod** validációt használ a robusztus biztonság érdekében.

A szerveroldali technológiák kiválasztásánál a fő szempont az volt, hogy a backend elég gyors, skálázható és biztonságos legyen a valós idejű chat és az érzékeny személyes adatok (okmányok) kezeléséhez. Az Orion backendje ezért a Node.js és az Express környezetére épül, szorosan integrálva a Supabase ökoszisztémával.

4.2.1. Miért pont Node.js és Express 5?

Egy olyan rendszernél, ahol a frontend és a backend is ugyanabban a nyelvben (TypeScript) íródik, a kódmegosztás és a fejlesztői kontextusváltás (context switching) minimalizálása hatalmas előny. A Node.js aszinkron, eseményvezérelt architektúrája (Event Loop) tökéletes a sok párhuzamos, de I/O intenzív feladathoz, mint amilyen a hirdetések lekérése vagy az üzenetek továbbítása.

Bár használhattunk volna nehezebb keretrendszereket (pl. NestJS), az Express egyszerűsége és flexibilitása sokkal jobban illeszkedik az Orion moduláris (route-service) felépítéséhez. Az Express 5 ráadásul beépítve támogatja az aszinkron hibakezelést (Promise-based routing), így kevesebb try-catch blokkra van szükség, és a kód átláthatóbb marad.

4.2.2. Biztonság, Validáció és Hitelesítés

Az adatok integritása kritikus. A bejövő kérések (HTTP POST/PATCH) validálására ugyanazt a **Zod** könyvtárat használjuk a backend-en is, mint a frontend-en. Ez garantálja, hogy hibás vagy hiányos adat soha ne juthasson el az adatbázis rétegig.

Az autentikáció saját fejlesztésű, **JWT (JSON Web Token)** alapú rendszer. A tokeneket szigorúan, httpOnly cookie-kban adjuk át, így védve a rendszert a kliensoldali támadásoktól (XSS). A jelszavakat **bcrypt** algoritmussal hasheljük, így adatbázis-szivárgás esetén is visszafejthetetlenek maradnak.

4.2.3. Adatbázis és Realtime Képességek (Supabase)

A hagyományos adatbázisok helyett a **Supabase (PostgreSQL)** platformra építkezünk. Ez egyszerre ad egy sziklaszilárd, relációs adatbázist (PostgreSQL) és egy modern API réteget. A Supabase segítségével a bonyolult üzleti logika egy részét (például a statisztikák számítását) közvetlenül az adatbázisba, tárolt eljárásokként (RPC - Remote Procedure Call) tudtuk kiszervezni.

A chat funkció elengedhetetlen része egy modern diákmunka platformnak. A Supabase beépített Realtime modulját (amely PostgreSQL triggerekre épül) egy saját RealtimeServer WebSocket komponenssel kötöttük össze az Express-en belül, így az üzenetek másodpercek töredéke alatt, frissítés nélkül megérkeznek a kliensekhez.

4.2.4. Fájlkezelés és Értesítések

A diákok önéletrajzainak (PDF) biztonságos feltöltését a szerveren a **Multer** kezeli, mielőtt azokat a Supabase titkosított Storage rendszerébe (bucket) továbbítaná. Az automatizált értesítésekhez, mint például a regisztráció megerősítése vagy az elfelejtett jelszó visszaállítása, a **Nodemailer** könyvtárat integráltuk (Gmail SMTP használatával).

4.2.5. Backend Technológiai Verem Összefoglaló

Kategória	Technológia	Leírás / Szerepkör
Futtatókörnyezet	Node.js	V18+ aszinkron futtatókörnyezet V8 motorral.
Keretrendszer	Express 5	Könnyűsúlyú, middleware-alapú webservert aszinkron routinggal.
Nyelv	TypeScript	Szerveroldali típusbiztonság, egységes nyelv a frontennel.
Adatbázis & Backend-as-a-Service	Supabase (PostgreSQL)	Relációs DB, RPC függvények, Storage és pgcrypto titkosítás.
Hitelesítés (Auth)	JWT & bcryptjs	Állapotmentes (stateless), httpOnly cookie-alapú azonosítás és jelszó hashelés.
Adatvalidáció	Zod	Séma alapú futásidejű ellenőrzés a POST/PATCH kéréseknél.
Valós Idejű Kom.	ws (WebSocket)	Natív WebSocket szerver a chatüzenetek (RealtimeServer) közvetítésére.
Fájlkezelés	Multer	Multipart/form-data kérések és CV feltöltések pufferelt kezelése.
E-mail küldés	Nodemailer	SMTP alapú tranzakcionális emailek (aktiválás, jelszó csere).
AI Integráció	Python & FastAPI	Mikroszolgáltatás (OrionAI.py) NLP alapú álláskereséshez.

4.3. A választott kombináció előnyei

Döntés	Előny	Orion szempontjából jelentősége
React + TypeScript	Komponens-alapú fejlesztés, típusbiztonság	Nagyobb kódbázisnál könnyebb karbantartás és kisebb regressziós kockázat
Vite	Gyors fejlesztői szerver és build	Gyors iteráció UI-centrikus fejlesztés során
Express	Jól érthető route-modell és széles ökoszisztéma	Átlátható doménalapú szervermodulok
Supabase / PostgreSQL	Erős relációs modell, RPC, realtime, pgcrypto	Jelentkezések, chat és érzékeny dokumentumok miatt különösen előnyös
React Hook Form + Zod	Egységes űrlapkezelés és validáció	Regisztráció, profilkezelés, hirdetés- és jelentkezési űrlapok konzisztenciája

5. Adatmodell és adatbázislogika

5.1. Adatbázis és Technológiai Háttér (Supabase / PostgreSQL)

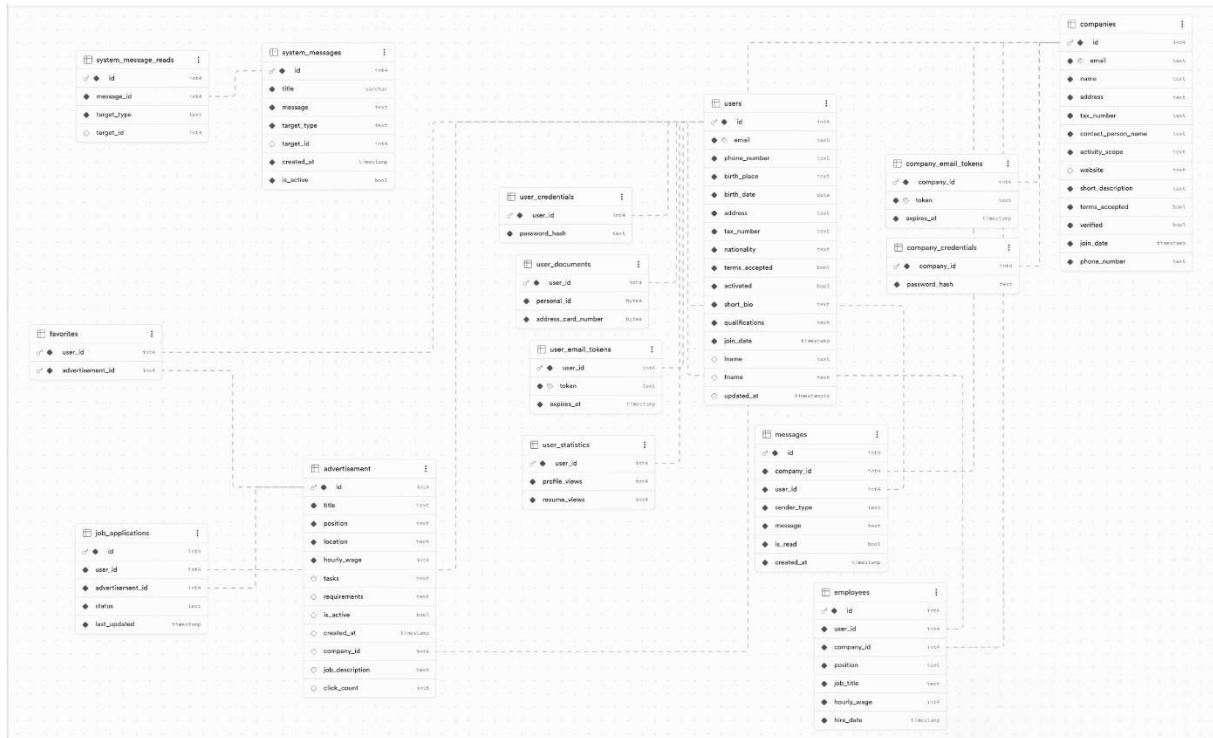
A projekt adatbázis-motorjaként a **Supabase** platformot választottuk, amely egy megbízható **PostgreSQL** relációs adatbázisra épül. A választás főbb okai a beépített biztonsági funkciók, az egyszerű kezelhetőség és a gyors működés voltak. A rendszer a következő funkciókat használja ki a legjobban:

- **Megbízható adattárolás:** A PostgreSQL biztosítja, hogy az adatok (pl. állásjelentkezések, munkaviszonyok) konzisztensek maradjanak, és esetleges hiba esetén se sérüljenek vagy vesszenek el (tranzakciókezelés).
- **Beépített titkosítás (pgcrypto):** A GDPR előírások miatt a legérzékenyebb személyes adatokat (személyi igazolvány, lakcímkártya) nem tároljuk olvasható szöveggént. Az adatbázis pgcrypto bővítménye segítségével ezeket az adatokat már közvetlenül az adatbázis szintjén titkosítva mentjük el.
- **Szerveroldali függvények (RPC):** A bonyolultabb számításokat (pl. statisztikák lekérése) és a több lépésből álló feladatokat (pl. biztonságos regisztráció) közvetlenül az adatbázisban, úgynevezett tárolt eljárásokban (RPC) futtatjuk. Ezzel gyorsítjuk a rendszert, mert kevesebb adatot kell mozgatni a szerver és az adatbázis között.
- **Valós idejű kommunikáció (Real-time):** A Supabase beépített WebSocket megoldásának köszönhetően a chat üzenetek azonnal megjelennek a feleknél, anélkül, hogy a böngészőt folyamatosan frissíteni kellene, vagy a szervert terhelnénk a folyamatos lekérdezésekkel.

Kiterjesztés telepítése: Az adatbázis első beállításakor futtatni kell a `CREATE EXTENSION IF NOT EXISTS pgcrypto;` parancsot az SQL felületen, mert ezen múlik az okmányok titkosított tárolása.

5.2. Adatbázis sémakép

Az alábbi ábra szemlélteti a rendszer teljes adatbázis-architektúráját, a táblák közötti kapcsolatokat és a referenciális integritási szabályokat.



5.3. Táblák részletes leírása

Az adatbázis négy fő logikai modulra bontható: **Felhasználók**, **Cégek**, **Álláshirdetések** / **Munkaviszony** és **Kommunikáció** / **Üzenetek**.

5.3.1. Felhasználói (Diák) Modul

1. users (Álláskereső Törzstáblája)

Ez a tábla tárolja az álláskeresők alapvető személyes adatait és a profiljukhoz tartozó leírásokat. A biztonság jegyében a személyazonosító okmányokat külön kezeljük: ezek nem itt, hanem a titkosított user_documents táblában találhatók.

users	
id	int4
email	text
phone_number	text
birth_place	text
birth_date	date
address	text
tax_number	text
nationality	text
terms_accepted	bool
activated	bool
short_bio	text
qualifications	text
join_date	timestamp
lname	text
fname	text
updated_at	timestampz

2. user_credentials (Hitelesítő Adatok)

A felhasználók belépési adatainak biztonságos tárolója. A minimális jogosultság elve alapján a bejelentkezési adatok elvannak szeparálva a profiladatokról. Itt kizárólag a bcrypt algoritmussal képzett jelszó-hashek tárolódnak.

user_credentials	
user_id	int4
password_hash	text

3. user_documents (Titkosított Okmánytár)

Ez a tábla az érzékeny adatok védelmére szolgál, és fizikailag is elkülönítve tárolja az álláskeresők igazolványait a profiladatokról. A GDPR elveinek megfelelően az adatok nem nyílt szöveggént, hanem a pgcrypto bővítmény segítségével, PGP-alapú szimmetrikus titkosítással kerülnek a rendszerbe BYTEA formátumban. Így az okmányok száma csak a megfelelő kulcs birtokában fejthető vissza.

user_documents	
user_id	int4
personal_id	bytea
address_card_number	bytea

4. user_email_tokens (Tranzakciós Tokenek)

Ez a tábla kezeli a felhasználói fiókok megerősítéséhez és a jelszó-visszaállításhoz szükséges egyszer használatos azonosítókat. Minden rekordhoz kötelező lejáratási idő (TTL) tartozik, amely biztosítja, hogy a kiküldött kódok csak meghatározott ideig maradjanak érvényesek, növelve ezzel a rendszer biztonságát.

user_email_tokens	
user_id	int4
token	text
expires_at	timestamp

5. user_statistics (Interakciós adatok)

Ez a tábla az állás keresők profiljához kapcsolódó aktivitásokat rendszerezi, külön mérve a profil adatok és az önéletrajzok megtekintéseinek számát. Az adatok integritását egy adatbázis-trigger biztosítja, amely minden új regisztrációkor automatikusan létrehozza a felhasználóhoz tartozó, alaphelyzetbe állított (nullázott) statisztikai rekordot.

user_statistics	
user_id	int4
profile_views	int4
resume_views	int4

6. favorites (Kedvencek)

Ez a kapcsolótábla teszi lehetővé az állás keresők számára a hirdetések mentését és későbbi gyors elérését. A tábla egy több-a-többhöz kapcsolatot valósít meg a felhasználók és az álláshirdetések között. Az összetett elsődleges kulcs garantálja, hogy egy felhasználó csak egyszer jelölhet meg kedvencként egy adott hirdetést.

favorites	
user_id	int4
advertisement_id	int4

5.3.2. Vállalati (Cég) Modul

7. companies (Vállalati Törzstábla)

A hirdető partnerek adminisztratív és profiladatainak tárolója. A tábla külön kezeli a kapcsolattartó személyét és a cég hivatalos elérhetőségeit.

companies	
id	int4
email	text
name	text
address	text
tax_number	text
contact_person_name	text
activity_scope	text
website	text
short_description	text
terms_accepted	bool
verified	bool
join_date	timestamp
phone_number	text

8. company_credentials (Céges Hitelesítő Adatok)

A cégek belépési jelszavai, a user_credentials táblához hasonló elven tárolva a maximális szeparáció érdekében.

company_credentials	
company_id	int4
password_hash	text

9. company_email_tokens (Céges Tokenek)

Ez a tábla kezeli a munkáltatói fiókok megerősítéséhez és a jelszó-visszaállításhoz szükséges egyszer használatos azonosítókat. Minden rekordhoz kötelező lejáratási idő (TTL) tartozik, amely biztosítja, hogy a kiküldött kódok csak meghatározott ideig maradjanak érvényesek, növelve ezzel a rendszer biztonságát.

company_email_tokens	
company_id	int4
token	text
expires_at	timestamp

5.3.3. Hirdetések és Munkaviszony (ATS Modul)

10. advertisement (Álláshirdetések)

Ez a tábla a rendszerben közzétett álláslehetőségek központi jegyzéke. Tartalmazza a munkakör pontos megnevezését, a bérsávot, valamint a pozíció betöltéséhez szükséges elvárásokat és feladatokat. A hirdetések közvetlen kapcsolatban állnak a hirdető vállalattal, az is_active státusz pedig lehetővé teszi a hirdetések láthatóságának azonnali kezelését. A népszerűségi mutatók nyomon követésére a tábla natív módon vezeti a megtekintések számát, segítve ezzel a hirdetések hatékonyságának elemzését.

advertisement	
id	int4
title	text
position	text
location	text
hourly_wage	int4
tasks	text
requirements	text
is_active	bool
created_at	timestamp
company_id	int4
job_description	text
click_count	int8

11. job_applications (Pályázatok)

Ez a tábla kezeli az álláskeresők és a hirdetések közötti kapcsolatot, rögzítve minden egyes pályázati folyamatot. Nem csupán egy egyszerű összekötő elem, hanem egy állapotkövető (State Tracker) rendszer, amely a status mező segítségével transzparensen mutatja a jelentkezés aktuális szakaszát (pl. benyújtva, elbírálás alatt, elfogadva). A last_updated időbélyeg biztosítja a folyamatok időbeliségének nyomon követhetőségét, így mindig látható, mikor történt az utolsó változtatás a pályázat sorsában.

job_applications	
id	int4
user_id	int4
advertisement_id	int4
status	text
last_updated	timestamp

12. employees (Alkalmazotti Nyilvántartás)

Ez a tábla a sikeres kiválasztási folyamat lezárásaként létrejött aktív munkaviszonyok hivatalos jegyzéke. Nem csupán statisztikai adat, hanem a diák és a munkáltató közötti szerződéses feltételek pontos, visszakereshető lenyomata. Rögzíti a betöltött pozíciót, a megállapodás szerinti órabért és a munkába állás pontos időpontját. A tábla biztosítja a jogfolytonosságot és az adminisztratív átláthatóságot a teljes munkaviszony alatt.

employees		
id	int4	
user_id	int4	
company_id	int4	
position	text	
job_title	text	
hourly_wage	int4	
hire_date	timestamp	

5.3.4. Kommunikáció és Értesítési Rendszer

13. messages (Chat Motor)

Ez a tábla a rendszer üzenetküldő motorjának központi tárolója, amely biztosítja a párbeszéd folyamatosságát a diákok és a munkáltatók között. A WebSocket-alapú valós idejű csevegés háttérbázisaként szolgál, így az üzenetváltások később is bármikor visszakereshetők és megtekinthetők. A rendszer biztonságát a `sender_type` mezőre illesztett adatszintű ellenőrzés (CHECK constraint) garantálja, amely megakadályozza a feladói szerepkörök felcserélését. A tábla emellett megbízhatóan rögzíti az üzenetek olvasottságát és a küldés pontos idejét.

messages		
id	int4	
company_id	int4	
user_id	int4	
sender_type	text	
message	text	
is_read	bool	
created_at	timestamp	

14. system_messages (Rendszerüzenetek)

Ez a tábla az adminisztrátorok által küldött hivatalos értesítések és globális üzenetek gyűjtőhelye. A rugalmas felépítés lehetővé teszi, hogy az üzeneteket pontosan célozzuk meg: küldhetők minden felhasználónak, vagy akár specifikusan csak egy adott csoportnak (pl. csak a cégeknek vagy csak a diákoknak). A tábla biztosítja, hogy a fontos rendszerüzenetek — mint például a karbantartási tájékoztatók vagy jogi frissítések — rendezett formában jussanak el a címzettekhez.

system_messages		
id	int4	
title	varchar	
message	text	
target_type	text	
target_id	int4	
created_at	timestamp	
is_active	bool	

15. system_message_reads (Rendszerüzenet Megtekintések)

Ez a tábla a rendszerüzenetek megtekintéseit követi nyomon, biztosítva a felhasználói élmény zavartalanságát. Kizárólag azt rögzíti, ha egy adott üzenetet a címzett már elolvasott, így a kezelőfelület (frontend) pontosan tudja, mely értesítéseket nem kell többé megjeleníteni. Ez a megoldás megakadályozza a redundáns információközlést, és segít tisztán tartani a felhasználók értesítési listáját.

system_message_reads	
id	int4
message_id	int4
target_type	text
target_id	int4

5.4. Referenciális Integritás (ER Összefoglaló)

Az adatbázis kiterjedten alkalmazza a FOREIGN KEY ... ON DELETE CASCADE szabályokat. Ha egy entitás (pl. egy Cég) törlődik a platformról, az adatbázis motorja automatikusan – alkalmazásszintű logika bevonása nélkül – eltávolítja a cég összes hirdetését, munkavállalói kapcsolatait, hitelesítő adatait és chat előzményeit, meggátolva az "árva" (orphan) rekordok kialakulását.

5.5. Szerveroldali Tárolt Eljárások (RPC)

Az Orion a komplex tranzakciókat és aggregációs számításokat közvetlenül a PostgreSQL motoron belül végzi (PL/pgSQL nyelven). Minden érzékeny függvény SECURITY DEFINER futási kontextust használ, felülbírálvá az alapértelmezett jogosultságokat a biztonság érdekében. A rendszerben 13 dedikált eljárás (RPC) dolgozik:

- 1. `register_user_v1(...)`: Tranzakcionális felhasználói regisztráció. Egy lépésben (atomi módon) hozza létre a profilt, menti a jelszó-hashet, és (az `insert_encrypted_documents` hívásával) titkosítva tárolja a diák okmányait. Hiba esetén a teljes folyamat automatikus ROLLBACK-et kap.
- 2. `register_company_v1(...)`: Tranzakcionális céges regisztráció, amely biztosítja a vállalat adatainak és belépési adatainak konzisztens, egyidejű mentését az adatbázisba.
- 3. `insert_encrypted_documents(...)`: A pgcrypto bővítményre építve szimmetrikusan titkosítja és menti az új személyi okmányokat.
- 4. `update_encrypted_documents(...)`: A meglévő titkosított okmányokat írja felül új értékekkel (pl. adatváltozás esetén).
- 5. `get_decrypted_documents(...)`: Kizárólagos kapu a szenzitív adatokhoz. Biztonságosan visszafejti az igazolványszámokat a paraméterként átadott titkosítási kulccsal. A kulcs magában az adatbázisban sehol nem tárolódik.

- 6. `get_user_dashboard_stats(...)`: Komplex lekérdezés, amely egyetlen JSON objektumban adja vissza a diák irányítópultjának összes KPI adatát (leadott/elfogadott jelentkezések, önéletrajz-letöltések, profilmegtekintések).
- 7. `get_company_dashboard_stats(...)`: A munkáltatói irányítópulthoz generál valós idejű statisztikákat, beleértve a hirdetésmegtekintéseket, a beérkezett jelentkezők számát, és az ezekből automatikusan számított konverziós arányokat.
- 8. `get_user_chat_partners(...)`: A végtelen görgetéshez (infinite scroll) optimalizált lekérdezés, amely visszaadja a diák minden aktív céges chatpartnerét a legutolsó elküldött üzenet időpontjával együtt (DISTINCT ON logika).
- 9. `get_company_chat_partners(...)`: Ugyanaz a funkcionalitás munkáltatói oldalon, amely utolsó interakció szerint csökkenő sorrendben rendezi a diákokat a céges chat felületen.
- 10. `increment_click_count(...)`: Atomi számláló, amely egy hirdetés megtekintésekor kiküszöböli a versengési helyzeteket (Race Conditions), pontosan növelve a látogatottságot.
- 11. `increment_profile_views(...)`: A diák adatlapjának látogatottságát regisztráló atomi RPC.
- 12. `increment_resume_views(...)`: A diák önéletrajzának (CV) letöltésekor lefutó atomi számláló.
- 13. `create_user_statistics(...)`: (Lásd lentebb a triggereknél) Egy inicializáló függvény, amely egy új diák statisztikáit hozza létre nullázott értékekkel.

5.6. Eseményvezérelt Adatbázis Logika (Triggerek)

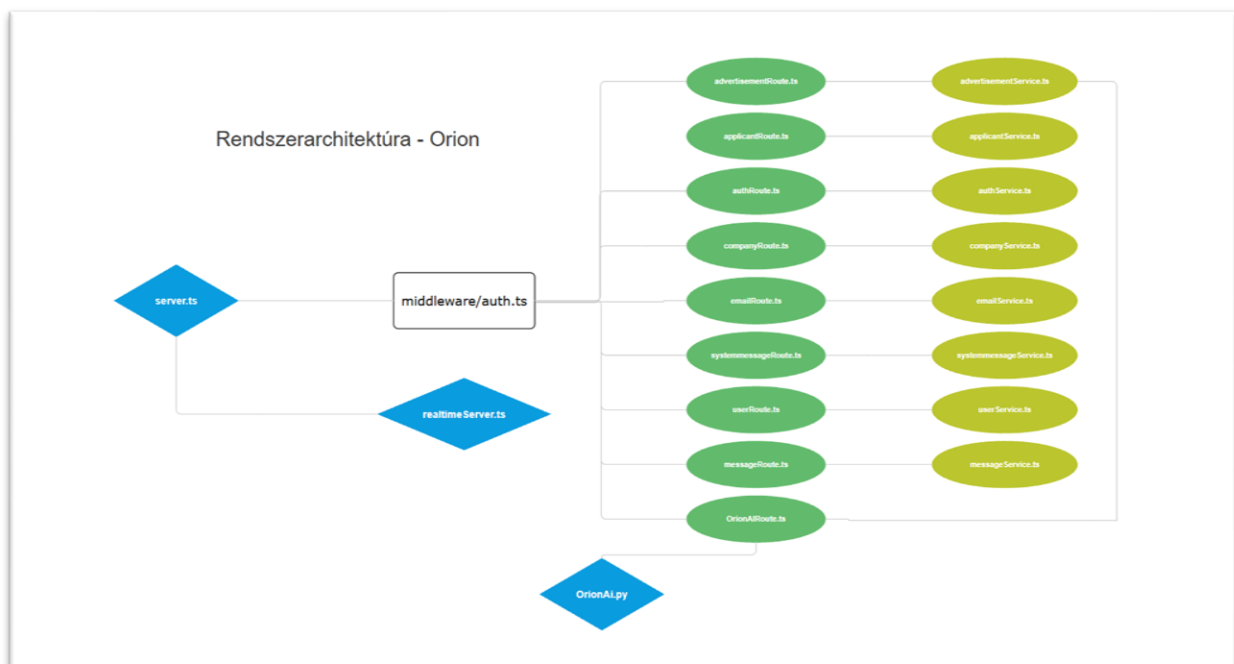
- `trg_create_user_statistics`: Egy AFTER INSERT esemény kiváltó, amely a `create_user_statistics` eljárást hívja meg. Amint egy új felhasználó regisztrál, aszinkron módon létrehozza a `user_statistics` sort. Biztosítja az adatszerkezet integritását anélkül, hogy az alkalmazásréteget terhelné.

6. Backend Szerver Architektúra (src/server)

TL;DR: A backend architektúra felel az Orion platform üzleti logikájának biztonságos és skálázható kiszolgálásáért. A **Node.js és Express 5** alapú REST API robusztus adatbázis-kapcsolatot tart fenn a Supabase-zel, miközben integrálja a valós idejű WebSocket kommunikációt (chat) és az egyedi NLP mikroszolgáltatást (OrionAI). Minden hálózati kommunikáció szigorú token-alapú hitelesítéssel és sémavalidációval védett.

A szerveroldali alkalmazás Node.js környezetben, TypeScript nyelven fut. A fő belépési pont a `server.ts` állomány, amely az Express alkalmazás teljes életciklusát menedzseli. Feladatai közé tartozik a globális middleware-ek (CORS, Cookie-parser, JSON test feldolgozás) regisztrálása, a moduláris útvonalak (Routes) csatolása, valamint a független WebSocket szerver inicializálása. A rendszer alapértelmezetten a 4000-es porton hallgat, és elkülöníti a hálózati réteget (Routes) az üzleti logikától (Services).

Az alábbi ábra szemlélteti a rendszer teljes Backend-architektúráját



6.1. Hálózati Middleware és Realtime Kommunikáció

A backend architektúrális alapelve, hogy a végpontok csak ellenőrzött és hitelesített kéréseket szolgálhatnak ki, illetve hogy a chat-funkció azonnal, HTTP polling nélkül működjön (ne kelljen a frontendnek folyamatosan kérdezgetni a szervert, hogy van-e új üzenet, illetve ne legyen késleltetés az üzenetek érkezésében és ne kelljen a böngészőt folyamatosan újratölteni a felületen).

auth.ts (Globális Hitelesítési Middleware)

A biztonsági kapu (Gateway), amely minden védett végpont előtt lefut, biztosítva a Zero-Trust modellt.

- **Működési folyamat:**
 1. **Token kinyerése:** A middleware a HTTP kérésből szigorúan a `httpOnly` flag-gel ellátott `auth_token` süti tartalmát olvassa ki. Ezzel a módszerrel a token immunis az XSS támadásokra.
 2. **Kriptográfiai ellenőrzés:** A `jsonwebtoken` könyvtár `verify` módszerével validálja a token szerveroldali aláírását és érvényességi idejét.
 3. **Adatinjektálás:** Sikeres ellenőrzés után a token dekódolt tartalmát (`userId`, `companyId`, `userType`) közvetlenül az Express Request objektumba ágyazza.
 4. **Integritásvizsgálat:** A specifikus `verifyUser` és `verifyCompany` middleware-ek extra ellenőrzést végeznek az adatbázissal, garantálva, hogy törölt vagy felfüggesztett fiókok ne kapjanak hozzáférést érvényes token birtokában sem.

RealtimeServer.ts (WebSocket Szolgáltatás)

A privát üzenetek késedelem nélküli (Zero-Latency) továbbításáért felelős önálló szolgáltatás, amely a HTTP szerver felett fut.

- **Működési folyamat:**
 1. **Inicializálás:** A `ws` (WebSocket) könyvtár használatával egy dedikált csatornát nyit a felcsatlakozó klienseknek, és hitelesíti a bejövő kapcsolatokat.
 2. **Eseményfigyelés:** Feliratkozik a Supabase adatbázis `postgres_changes` mechanizmusára. Folyamatosan figyeli a `messages` táblába érkező új (INSERT) adatbázis-eseményeket.
 3. **Broadcasting:** Amint egy új üzenet bekerül az adatbázisba (akár a REST API-n keresztül), a `RealtimeServer` azonnal elkapja a payload-ot, és rápumpálja azt az érintett kliensek nyitott WebSocket csatornájára. Ezzel tehermentesíti a klienst a felesleges API lekérdezésektől.

6.2. API Végpontok és Szolgáltatások (Routes & Services)

A rendszer felépítése úgy lett kialakítva, hogy hosszú távon is jól átlátható, karbantartható és könnyen bővíthető maradjon. Ennek érdekében a Domain-Driven Design (DDD) megközelítést alkalmazza, amely az alkalmazást különálló, logikailag elkülönített modulokra bontja (pl. autentikáció, felhasználók, cégek, hirdetések).

Ez a felosztás lehetővé teszi, hogy minden modul egy konkrét feladatért feleljen, így a fejlesztés során egyszerűbb új funkciókat hozzáadni vagy meglévőket módosítani anélkül, hogy az egész rendszert érintenénk.

Minden modul két jól elkülöníthető rétegből áll:

- **Route réteg (*Route.ts):** Ez a réteg felelős a kliens (pl. frontend alkalmazás) felől érkező HTTP kérések fogadásáért. Itt történik a végpontok (endpointok) definiálása, az adatok alapvető ellenőrzése (validáció), valamint a megfelelő szolgáltatás (service) meghívása. Fontos, hogy itt még nem történik komoly üzleti logika feldolgozás.
- **Service réteg (*Service.ts):** Ebben a rétegben található a rendszer „gondolkodása”. Itt valósul meg az üzleti logika, például adatbázis műveletek, szabályok ellenőrzése, számítások és egyéb folyamatok. A service réteg független a konkrét HTTP kérésektől, így más környezetben is újrahasználható.

A két réteg szétválasztása biztosítja, hogy a kód strukturált maradjon: a route csak a kommunikációért felel, míg a service a működésért. Ez nemcsak a fejlesztést gyorsítja, hanem a hibakeresést és a tesztelést is jelentősen megkönnyíti.

6.2.1. Autentikációs Modul (authRoute.ts / authService.ts)

A felhasználók és cégek életciklusának (regisztráció, belépés, kilépés) és hitelesítési folyamatainak biztonságos végrehajtása.

Végpont	Függvény	Architektúrális Felelősség
POST /Api/auth/user/login	loginUser	Diák fiók hitelesítése, jelszó ellenőrzése és JWT token generálása.
POST /Api/auth/user/register	registerUser	Új diák fiók létrehozása, jelszó hashelése és titkosított adatok elmentése.
POST /Api/auth/company/login	loginCompany	Cég fiók hitelesítése, jelszó ellenőrzése és JWT token generálása.
POST /Api/auth/company/register	registerCompany	Új munkáltatói fiók létrehozása, jelszó hashelése és adatbázisba elmentése.
POST /Api/auth/logout	–	Azonnali munkamenet-megszakítás a hitelesítő süti törlésével.
GET /Api/auth/status	verifyToken	A kliens számára biztosít végpontot az aktuális jogosultsági szint lekérdezésére.

- **Bejelentkezési (Login) folyamat:** Szigorú bemeneti validáció a Zod segítségével. Az email cím alapján a szerver lekéri a megfelelő fiókot, majd a bcrypt.compare használatával kriptográfiailag ellenőrzi a jelszó egyezését. Sikeres azonosítás után egy magas biztonságú, httpOnly JWT tokenet küld válaszként, amely beállítja a felhasználó munkamenetét.
- **Regisztrációs (Register) folyamat:** A jelszó azonnali hashelése (bcrypt) után a rendszer meghívja a Supabase RPC függvényeit (register_user_v1 / register_company_v1). Ez az eljárás garantálja a tranzakcionális adatbázis-műveleteket, így hiba esetén nem jöhet létre inkonzisztens vagy részleges adatprofil.

6.2.2. Felhasználói (Diák) Modul (userRoute.ts / userService.ts)

A diákok személyes profiljának, okmányainak (CV), kedvelt hirdetéseinek és platformon belüli statisztikáinak komplex kezelése.

Végpont	Függvény	Architektúrális Felelősség
GET /Api/users/me	getUserProfile	Személyes profil lekérése és az okmányok biztonságos, memórián belüli visszafejtése.
PATCH /Api/users/me	updateUserProfile	Profiladatok frissítése, újbóli titkosítási eljárások végrehajtásával.
POST /Api/users/me/resume	uploadResume	Egyedi PDF önéletrajz (max 5MB) puffertelt feltöltése a Supabase Storage bucketbe.
DELETE /Api/users/me/resume	deleteResume	Önéletrajz végleges eltávolítása a felhő alapú tárolóból.
GET /Api/users/me/stats	getUserDashboardData	Egységesített dashboard statisztikák (jelentkezések, nézettség) lekérdezése RPC-n keresztül.
GET /Api/users/me/resume	getResumeUrl	Kliens számára ideiglenes (lejárattal rendelkező) biztonságos letöltési link generálása.
POST /Api/users/me/favorites	addFavorite	Kapcsolótábla frissítése: álláshirdetés hozzáadása a személyes kedvencekhez.
GET /Api/users/me/favorites	getFavorites	A felhasználó által megjelölt preferált hirdetések lekérése.
DELETE /Api/users/me/favorites/:id	removeFavorite	Hivatkozás törlése a kedvencek listájáról.
POST /Api/users/:id/views	incrementProfileViews	Profil megtekintési mutatójának atomi növelése (RPC alapú statisztikagyűjtés).

- **Személyes adatok kezelése:** Az adatvédelem (GDPR) szempontjából kritikus adatok lekérésekor a szolgáltatás a get_decrypted_documents RPC függvényt hívja meg. A titkosított adatok visszafejtése közvetlenül az adatbázis memóriájában történik, így a hálózaton már csak a jogosult felhasználó számára dekódolt adatok utaznak.

- **Fájlkezelés (CV):** Az önéletrajzokat a rendszer normalizálja, kiterjesztés-ellenőrzésnek veti alá, majd egy biztonságos, jogosultság-vezérelt (RLS) Supabase Storage mappába tölti fel. A fájlok közvetlenül nem elérhetőek, lekérésükkor a rendszer úgynevezett *Signed URL*-eket generál, melyek rövid időn belül érvényüket veszítik, megakadályozva a dokumentumok kiszivárgását.

6.2.3. Cég Modul (`companyRoute.ts` / `companyService.ts`)

A munkáltatók publikus adatainak, irányítópultjának (dashboard) és a rendszeren keresztül felvett alkalmazottak adminisztrációjának központja.

Végpont	Függvény	Architektúrális Felelősség
GET <code>/Api/companies/me</code>	<code>getCompanyProfile</code>	A munkáltató alap- és bemutatkozó adatainak kigyűjtése az adatbázisból.
PATCH <code>/Api/companies/me</code>	<code>updateCompanyProfile</code>	Céges profilinformációk validált frissítése.
GET <code>/Api/companies/me/stats</code>	<code>getCompanyStats</code>	Fő KPI adatok (megtekintések, futó hirdetések, friss jelentkezések) komplex lekérdezése.
GET <code>/Api/companies/me/employees</code>	<code>getEmployees</code>	A céghez hivatalosan felvett diákok (munkavállalók) listájának lekérése.
DELETE <code>/Api/companies/me/employees/:id</code>	<code>deleteEmployee</code>	Munkaviszony lezárása, alkalmazott törlése és automatikus rendszerüzenet (értesítés) generálása.

- **Alkalmazottak kezelése:** A munkaviszonyok (`employees`) adatbázisszintű összekapcsolást jelentenek egy diák és egy cég között. Egy alkalmazott eltávolításakor a szolgáltatás nem csak a relációt szünteti meg, hanem tranzakcionálisan egy értesítő rendszerüzenetet (System Message) is generál az érintett diáknak, biztosítva a teljes transzparenciát.

6.2.4. Álláshirdetések (advertisementRoute.ts / advertisementService.ts)

A platform legfontosabb entitásának, az álláshirdetéseknél a teljes CRUD (Create, Read, Update, Delete) életciklus-kezelése és keresési motorja.

Végpont	Függvény	Architektúrális Felelősség
POST /Api/advertisements	createAdvertisement	Új hirdetés publikálása szigorú limit-ellenőrzéssel (maximum 3 aktív hirdetés cégenként).
GET /Api/advertisements	getAllAdvertisements	A publikus felületek számára elérhető aktív hirdetések gyors listázása.
GET /Api/advertisements/:id	getAdvertisementById	Egyetlen hirdetés minden részletre kiterjedő, céges adatokkal join-olt lekérése.
GET /Api/companies/me/advertisements	getCompanyAdvertisements	A cég saját belső listájának (aktív és inaktív) lekérése az irányítópulthoz.
PATCH /Api/advertisements/:id	updateAdvertisement	Meglévő hirdetés paramétereinek biztonságos szerkesztése.
PATCH /Api/advertisements/:id/status	updateAdvertisementStatus	Hirdetés láthatóságának (aktív/inaktív) kapcsolása a találati listákban.
GET /Api/advertisements/search	searchAdvertisements	Robusztus szerveroldali keresés, paraméteres szűrés és lapozás (pagination) kivitelezése.
GET /Api/advertisements/top3	getTopAdvertisements	A kezdőlap optimalizált betöltését segítő, limitált lekérdezés.
PATCH /Api/advertisements/:id/views	incrementClickCount	RPC-alapú atomi hirdetés-kattintás számláló növelése.

- **Keresési mechanizmus:** A searchAdvertisements funkció dinamikusan épít fel egy komplex SQL-szerű lekérdezést (Query Builder) a paraméterek alapján (pl. szöveges egyezés a címben, minimális bér, város megjelölése). A memóriaszivárgások elkerülése és a hálózati sávszélesség kímélése érdekében az adatokat mindig szigorú lapozással (Pagination) és limitálva küldi vissza a kliensnek.
- **Üzleti szabályok betartatása:** A hirdetés-létrehozásnál a backend kőbe vésett szabályként ellenőrzi, hogy a cég nem lépte-e túl a megengedett 3 aktív hirdetési limitet. Ha igen, a rendszer API-szinten elutasítja a kérést (HTTP 403), megelőzve a platform spam-elését.

6.2.5. Jelentkezések (applicantRoute.ts / applicantService.ts)

Az ATS (Applicant Tracking System) motorja, amely az állásokra beérkező pályázatokat, státuszaikat és az elbírálási folyamatokat menedzseli.

Végpont	Függvény	Architektúrális Felelősség
POST /Api/applications	submitApplication	A diák összekapcsolása a hirdetéssel (jelentkezés leadása), duplikáció szűréssel.
GET /Api/users/me/applications	getUserApplications	A diák saját jelentkezési előzményeinek lekérdezése az aktuális elbírálási státuszokkal.
GET /Api/advertisements/:id/applications	getApplicantsForAdvertisement	A cégek számára biztosítja a hirdetésre érkezett profilok és pályázatok listáját.
PATCH /Api/applications/:id/status	acceptApplication/rejectApplication	Döntéshozatal: Pályázat elfogadása (munkaviszony generálással) vagy elutasítása.
GET /Api/applications/:id/resume	getApplicantResumeUrl	Pályázó CV-jének letöltése (jogosultságvizsgálattal) és a megtekintés adminisztrálása.

- **Állapotgép (State Machine) logika:** A jelentkezések a submitted, accepted vagy rejected állapotok között mozognak. A backend szigorúan felügyeli az állapotátmeneteket. Egy elfogadás (Accept) komplex folyamatot indít el: ellenőrzi a cég jogosultságát, módosítja a jelentkezés státuszát, hivatalosan is alkalmazottként rögzíti a diákot az employees táblában, és értesítést küld számára – mindezt adatbázis tranzakció keretében.

6.2.6. Kommunikáció és Chat (messageRoute.ts / messageService.ts)

A felhasználók közötti privát üzenetváltások és a chatpartnerek hálózati listázásának API rétege.

Végpont	Függvény	Architektúrális Felelősség
GET /Api/messages/conversations	getUserChatPartners / getCompany...	A legutóbbi üzenetváltások alapján aggregálja az aktív beszélgetőpartnereket.
GET /Api/messages/conversations/:id	getChatMessages	Lekéri a konkrét chatelőzményt, és atomi módon beállítja az üzenetek "olvasott" státuszát.
POST /Api/messages	sendMessage	Új üzenetet rögzít a messages táblába, amely kiváltja a RealtimeServer adásszórását.

- **Optimalizált adatelérés:** Mivel egy chat platformnál az üzenetek száma exponenciálisan nőhet, a partnerek listázását nem a backend kódban, hanem optimalizált adatbázis RPC függvényekkel (get_user_chat_partners) végezzük. Amikor a kliens megnyit egy beszélgetést, az API nemcsak visszaküldi az előzményeket, hanem egy háttérfolyamatban azonnal is_read=true állapotúra frissíti a bejövő üzeneteket, szinkronban tartva az értesítési piktogramokat.

6.2.7. Rendszerüzenetek (systemmessageRoute.ts / systemmessageService.ts)

A platformon belüli automatikus, adminisztratív értesítések (pl. elutasítás, munkaviszony vége) disztribúciós központja.

Végpont	Függvény	Architektúrális Felelősség
GET /Api/notifications	getUserSystemMessages	A célzott felhasználónak szóló, még el nem olvasott értesítések kigyűjtése.
PATCH /Api/notifications/:id/read	markSystemMessageAsRead	Pivot-tábla frissítése: a rendszerüzenet olvasottként való elkönyvelése az adott kliensnél.

- **Hatékony elosztás (Broadcasting logika):** A rendszerüzenetek lehetnek személyre szabottak vagy globálisak (mindenkinek, vagy az összes cégnek szólóak). A backend egy külön olvasási pivot-táblát (system_message_reads) tart fenn. Így egyetlen globális üzenetet elég egyszer eltárolni, a rendszer csak a megtekintés tényét rögzíti felhasználónként, drasztikusan csökkentve az adatbázis méretét.

6.2.8. Email Hitelesítés & Jelszó (emailRoute.ts / emailService.ts)

A fiókok aktiválási életciklusát és a jelszó-visszaállítás biztonságos hálózati folyamatait kezeli Nodemailer integrációval.

Végpont	Függvény	Architektúrális Felelősség
POST /Api/auth/.../verify-email	send...ActivationEmail	Rövid lejáratú, egyedi JWT tokenet generál, és elküldi az aktiváló hivatkozást emailben.
PATCH /Api/auth/.../verify-email	activate...Account	A beérkező token validálása után a fiók adatbázis szintű aktiválása.
POST /Api/auth/.../forgot-password	send...PasswordResetEmail	Biztonságos jelszóvisszaállító token és instrukciók küldése a regisztrált email címre.
PATCH /Api/auth/.../reset-password	saveNew...Password	Token érvényesítése, új jelszó kriptográfiai hashelése és mentése.

- **Biztonsági megfontolások:** Az aktivációs és jelszó-visszaállító emailekben szereplő linkek egyszer használatos (One-Time) JWT tokeneket tartalmaznak. A tokeneket a backend az adatbázisban (UPSERT elv alapján) tárolja, így garantálja, hogy ha a felhasználó új emailt kér, az előző link automatikusan érvényét veszti. A tokenek beépített lejáratú idővel (TTL - Time to Live) rendelkeznek. Sikeres felhasználás után a tokeneket a rendszer azonnal megsemmisíti.

6.2.9. OrionAI Mikroszolgáltatás (OrionAIRoute.ts / OrionAI.py)

Az Orion technológiai innovációja: a hagyományos Node.js szerver és a Python-alapú mesterséges intelligencia motor integrációs pontja.

Végpont	Függvény	Architektúrális Felelősség
POST /api/OrionAI	OrionAI.py API hívás / getAdvertisementsByIds	A felhasználó természetes nyelvű promptjának továbbítása az AI szerverhez, majd a visszaadott azonosítók alapján az adatok lekérése a Supabase-ből.

Megjegyzés: A mikroszolgáltatás technikai hátterének és a NLP alapú keresőmotornak a részletes ismertetését lásd a 8.2. OrionAI Integráció (Python NLP) szekcióban.

6.3. Hibakezelési Stratégia (Error Handling)

A backend robusztus és egységes hibakezelési mechanizmussal védi a rendszer stabilitását és az adatok integritását:

- Központi Try-Catch Szerkezet:** Minden Express útvonalkezelő (route handler) szigorú try-catch blokkba van csomagolva, így a váratlan szerverhibák nem okozhatják a Node.js folyamat összeomlását.
- Bemeneti Validációs Hibák (Zod):** A hibásan beküldött vagy hiányos adatok már az üzleti logika elérése előtt fennakadnak, és a rendszer tiszta, 400 Bad Request JSON válaszban tájékoztatja a klienst a pontos hibáról.
- Kezelt Üzleti Kivételek:** Az olyan specifikus üzleti eseteket, mint a "Már töltöttél fel önéletrajzot" vagy "Nem megfelelő jogosultság", a rendszer felismeri, és dedikált 400, 401 vagy 403-as HTTP kóddal jelzi, hogy a kliensoldali Toast értesítések (UI) azonnal feldolgozhatják azokat.
- Váratlan Szerverhibák (500):** Adatbázis-szakadások vagy belső kódhibák esetén a rendszer a console.error segítségével logol a fejlesztőknek, de a kliens felé csak egy biztonságos, általános 500 Internal Server Error választ ad, megelőzve az infrastruktúra részleteinek kiszivárgását.

7. Frontend architektúra

TL;DR: A frontend **moduláris, funkció-alapú** felosztást követ, egyedi állapotkezeléssel és szigorú, többszintű hozzáférésvédelemmel. A kód olyan üzleti területek (például auth, chat, public, company) köré szerveződik, ami lehetővé teszi a könnyű bővíthetőséget, a tiszta adatáramlást és az optimális teljesítményt anélkül, hogy a globális állapot feleslegesen bonyolódna.

7.1. Könyvtárstruktúra (Directory Structure)

Az Orion nem a megszokott módon (ahol minden gomb egy helyen, minden logika egy másik helyen van) épül fel. Ehelyett funkciók szerint csoportosítunk (Feature-based). Ez azért jó, mert ha például az Üzenetküldőn (OrionMessenger) dolgozunk, minden szükséges fájlt egy helyen találunk, és nem kell tíz különböző mappát átnéznünk. Így a kód sokkal átláthatóbb marad (cohesion), és később is könnyebben bővíthető. A projekt tényleges mappaszerkezete az alábbi logikát követi:

```
src/
├── Api/                # Globális API hívások (native fetch)
├── components/         # Újrahasznosítható UI elemek csoportosítva
│   ├── base/          # Nagyobb űrlap és profil komponensek (pl. LoginPage)
│   ├── layout/        # Elrendezési elemek (Header, Footer, Navigáció)
│   └── ui/            # Atomi dizájn elemek (Gombok, Beviteli mezők, Modalok)
├── features/          # Funkció-alapú modulok (Feature-based)
│   ├── auth/          # Autentikációs folyamatok (Belépés, Regisztráció)
│   ├── chat/          # Messenger és valós idejű kommunikáció
│   ├── company/       # Munkaadói irányítópult és toborzás
│   ├── public/        # Publikus nézetek (Pl. Főoldal)
│   └── user/          # Álláskeresői profil és jelentkezések
├── hooks/             # Egyedi React hook-ok
├── locales/           # Többnyelvűségi JSON fájlok (i18n)
├── test/              # Tesztelési környezet és segédfájlok
└── validation/        # Zod adatsémák és űrlap validációk (pl. Validation.ts)
```

7.2. Komponensek és felelősségek

A felületek építésénél különválasztjuk az "okos" (Smart) és az "egyszerű" (Dumb/Presentational) komponenseket. Az okos komponensek kezelik az adatokat és az üzleti logikát, míg az egyszerűek (mint egy gomb vagy egy beviteli mező) csak a megjelenítésért felelősek. Ez a szigorú szétválasztás segít abban, hogy az Orion könnyen fejleszthető és tesztelhető maradjon.

7.3. Ki mit láthat az oldalon? (Access Control)

Az oldalvédelmet egy központi megoldással (RBAC – Role Based Access Control) kezeljük. Az `IsLoggedIn` nevű elem folyamatosan figyeli, hogy ki van bejelentkezve, és milyen joga van. Ez biztos védelmet (protection) ad, hogy senki ne láthasson olyan

oldalakat, amikhez nincs jogosultsága (például egy álláskereső ne juthasson be a cégek kezelőfelületére).

```
// Részlet az IsLoggedIn.tsx implementációjából
function IsLoggedIn({ children, mode = "user" }: Props) {
  const [loading, setLoading] = useState(true);
  const [loggedIn, setLoggedIn] = useState(false);
  const [userType, setUserType] = useState<"user" | "company" | null>(null);

  useEffect(() => {
    const check = async () => {
      try {
        const data = await checkAuthStatus();
        setLoggedIn(Boolean(data.loggedIn));
        setUserType(data.userType || null);
      } finally {
        setLoading(false);
      }
    };
    check();
  }, []);

  if (loading) return <div>Betöltés...</div>;
  // Szerepkör alapú redirect logika...
}
```

Fontos tudni, hogy az RBAC védelem a frontend oldalán csak a felhasználói élményt javítja (UX Security): segít abban, hogy a felhasználó ne lásson zavaró, számára szükségtelen menüket. A valódi biztonságot a backend szerver végzi a JWT tokenek ellenőrzésével minden egyes kérésnél.

7.4. Az oldal részei és funkciói (Functional Decomposition)

Az Orion platform feature-alapú (feature-based) architektúrát követ: a frontend komponensek a `src/features` mappában, modulonként külön könyvtárakba szervezve találhatók. Az alábbi alfejezetek modulonként, egységes táblázatos formában mutatják be az egyes oldalak célját és adathálózati kapcsolataikat.

7.4.1. Feature modulok áttekintése

Feature	Felelősségi kör	Tartalom
<code>public</code>	Nyilvános felületek és belépési pontok	Kezdőlap, marketing tartalom, nyilvános álláslisták

Feature	Felelősségi kör	Tartalom
auth	Hitelesítéshez kapcsolódó nézetek	Bejelentkezés, regisztráció, jelszóvisszaállítás, email verifikáció
company	Céges működés és adminisztráció	Dashboard, profil, hirdetések, ATS, jelöltkezelés
user	Diák oldali munkakeresési életciklus	Profil, jelentkezések, önéletrajz, álláskeresés
chat	Közvetlen kommunikáció	Messenger, partnerlista, üzenetfolyam, valós idejű frissülés

7.4.2. Publikus Modul — `src/features/public`

A nem bejelentkezett felhasználók számára szolgáltatott felületek. Technikai fókusz: gyors betöltési idő (Lighthouse 100%) és reszponzív, SEO-barát megjelenés.

Oldal / Komponens	Cél	Adatkapcsolat
PublicHomePage	Platform bemutatása, regisztráció ösztönzése, funkciók (álláskeresés, toborzás) ismertetése	GET <code>/api/advertisements/top3</code>

7.4.3. Globális Autentikációs Modul — `src/features/auth`

Szerepkörtől független hitelesítési folyamatok (jelszókezelés, email verifikáció). A konkrét végpontok szerepkörönként eltérnek (user / company).

Oldal / Komponens	Cél	Adatkapcsolat
PasswordReset	Elfelejtett jelszó megújítása: kérő email kiküldése, majd új jelszó mentése token alapján	POST /api/auth/{szerepkör}/forgot- password PATCH /api/auth/{szerepkör}/reset- password
VerifyPage	Regisztrált email cím megerősítése token alapján	PATCH /api/auth/{szerepkör}/verify- email
SendVerifyPage	Verifikációs email újraküldése	POST /api/auth/{szerepkör}/verify- email

7.4.4. Munkaadói (Céges) Modul — src/features/company

Cégek teljes életciklusa: regisztrációtól a jelöltkezelésig. Az összetett űrlapoknál (hirdetésfeladás) React Hook Form + Zod validáció biztosítja a gépelés közben is gyors felületet. Az ATS Kanban-nézet Framer Motion animációkat használ a státuszváltások megjelenítésére.

Oldal / Komponens	Cél	Adatkapcsolat
LoginPage	Céges felhasználók beléptetése	POST /api/auth/company/login

Oldal / Komponens	Cél	Adatkapcsolat
RegisterPage	Új cég regisztrációja	POST /api/auth/company/register
HomePage	Céges dashboard, alapvető statisztikák áttekintése	GET /api/companies/me/stats GET /api/companies/me/advertisements
EditProfile	Céges profil adatainak karbantartása	GET /api/companies/me PATCH /api/companies/me
ShowListedJobs	Aktív/inaktív hirdetések áttekintése	GET /api/companies/me/advertisements
CompanyJobs	Új munkakör kiírása, meglévő szerkesztése, státuszváltás	POST /api/advertisements GET /api/advertisements/{id} PATCH /api/advertisements/{id} PATCH /api/advertisements/{id}/status
ATS felület	Jelentkezők listázása, státusz frissítése (elfogadás/elutasítás) , önéletrajz letöltése	GET /api/advertisements/{id}/applications PATCH /api/applications/{id}/status GET /api/applications/{id}/resume

7.4.5. Álláskeresői (Diák) Modul — src/features/user

Diákok és álláskeresők számára biztosított felületek, a regisztrációtól a jelentkezések visszakövetéséig. Technikai kihívás: az OrionAI alapú intelligens keresés integrációja, valamint az aszinkron lapozás (pagination) memóriaszivárgás-mentes kivitelezése.

Oldal / Komponens	Cél	Adatkapcsolat
LoginPage	Diákok/álláskeresők beléptetése	POST /api/auth/user/login
RegisterPage	Diák regisztráció (lakcímkártya, személyi adatok)	POST /api/auth/user/register
UserJobs	Hirdetések listázása, részletes adatlap, mentett/kedvenc állások kezelése	GET /api/advertisements/search GET /api/advertisements/{id} GET /api/users/me/favorites POST /api/OrionAI
EditProfile	Felhasználói profil szerkesztése, önéletrajz megtekintése/törlése	GET /api/users/me PATCH /api/users/me GET /api/users/me/resume DELETE /api/users/me/resume
JobApplications	Benyújtott jelentkezések visszakövetése	GET /api/users/me/applications
UploadResume	Önéletrajz (PDF) feltöltése, megtekintése, törlése	POST /api/users/me/resume GET /api/users/me/resume DELETE /api/users/me/resume

7.4.6. Kommunikációs Modul — `src/features/chat`

Valós idejű, közvetlen üzenetküldés munkaadók és jelöltek között. Technikai kihívás: az "Optimistic UI" technika alkalmazása — az elküldött üzenet azonnal megjelenik a felületen, még a szerver visszaigazolása előtt, majd szükség esetén visszagörgetés

történik hiba esetén. Az állapotot a globális `MessengerProvider` (Context API) menedzseli, amely REST API hívásokkal és WebSocket kapcsolattal (`ws://localhost:4000`) tartja fenn a valós idejű szinkronizációt.

Oldal / Komponens	Cél	Adatkapcsolat
OrionMessenger	Partnerlista lekérése	GET <code>/api/messages/conversations</code> (user) GET <code>/api/messages/conversations/company</code> (company)
	Üzenetek lekérése és küldése	GET <code>/api/messages/conversations/{id}</code> POST <code>/api/messages</code> WebSocket <code>ws://localhost:4000</code>

7.5. Adatkezelés: Hogyan mozognak az adatok? (State Management & Data Flow)

Nagy hiba lenne minden adatot egyetlen közös helyen tárolni, mert akkor minden apró változástól (például egy mezőbe való gépeléstől) az egész oldal újrafrissülne és lelassulna. Az Orionban ezért szigorúan szétválasztjuk a helyi (Local) és a globális (Global) adatokat.

Az olyan ideiglenes (transient) dolgokat, mint egy lenyíló menü állapota vagy egy űrlap szövege, helyben kezeljük a komponenseken belül (`useState`). Csak a legszükségesebb adatokat tesszük közösbe (Context API), amikre sok helyen szükség van: például a bejelentkezési adatok (JWT tokenek, szerepkörök) vagy az Üzenetküldő (OrionMessenger) rendszer állapota.

Az adatok mozgása az Orionban mindig egyirányú (Unidirectional Data Flow). Ez segít abban, hogy a rendszer kiszámítható és stabil maradjon. Amikor a felhasználó például beküld egy űrlapot, az adat végigmegy bizonyos lépéseken: a gombnyomástól a hálózati lekéréseken át egészen a képernyő frissítéséig.

Adatáramlási Modell (Data Flow)

```
[User UI] (Click Application)
  ↓
[Smart Component] (Handle Submit)
  ↓
```

```
[API Service] (src/Api/applicationApi.ts)
  ↓
[Native Fetch] (Credentials: include)
  ↓
[Backend] (Process JWT & Logic)
  ↓
[UI Update] (Reconciliation)
```

JWT Munkamenet-ellenőrzés (Auth Flow)

```
[Browser] (Request to /auth/status)
  ↓ (Automatikus 'auth_token' cookie küldés)
[Express Server] (authRoute.ts)
  ↓ (jwt.verify(token, secret))
[Supabase] (Fetch User/Company metadata)
  ↓ (JSON Response)
[Frontend] (Update IsLoggedIn state)
```

Az adatok áramlása az Orion architektúrájában szigorúan egyirányú. A kliensoldali állapotmegőrzés a httpOnly cookie-kra épül, ami maximális biztonságot garantál az XSS támadásokkal szemben.

```
// Példa: Autentikációs állapot ellenőrzése (authApi.ts)
export const checkAuthStatus = async (): Promise<AuthStatus> => {
  const res = await fetch(`${API_BASE_URL}/auth/status`, {
    method: "GET",
    credentials: "include",
    cache: "no-store",
  });
  if (!res.ok) throw new Error("Failed to check auth status");
  return res.json();
};
```

Ez a determinisztikus adatkérési életciklus biztosítja, hogy a vizuális keretrendszer soha nem tartalmaz adathálózati specifikus "spagetti-kódot".

7.6. Űrlapok és adatok ellenőrzése (Form Handling & Validation)

A nagyobb adatbeviteli felületeknél (például a profil szerkesztésénél vagy álláshirdetés feladásánál) fontos, hogy az oldal ne akadjon meg gépelés közben. Ehhez egy modern technikát (react-hook-form) használunk, ami segít gyorsan tartani a felületet.

Az adatok helyességét egy sémavalidációs rendszerrel (Zod) ellenőrizzük. Ez lehetővé teszi, hogy még azelőtt figyelmeztessük a felhasználót a hibákra, hogy az adatok elhagynák a böngészőt. Így azonnali visszajelzést kap, ha például elfelejtett kitölteni egy kötelező mezőt.

```
// Diák/User regisztráció sémája (Validation.ts)
export const userRegisterObject = authBase.extend({
  lname: z.string().min(2, "Vezetéknév legalább 2 karakter").regex(/^[a-zA-ZáéíóöőúüűÁÉÍÖŐÚÚÜÜ\S]+$/, "A mező csak betűket tartalmazhat"),
```



```

    fname: z.string().min(2, "Keresztnév legalább 2 karakter").regex(/^[a-zA-ZáéíóöőúüűÁÉÍÓÖŐÚÚÚ\ſ]+$/, "A mező csak betűket tartalmazhat"),
    birth_place: z.string().min(2, "Születési hely legalább 2 karakter"),
    birth_date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/, "Érvénytelen dátum").refine((date) => {
      return new Date(date) <= new Date();
    }, "A születési dátum nem lehet a jövőben"),
    // ...
  });

```

A komplex, több képernyőt érintő folyamatokat (mint a Messenger rendszere) dedikált Context Provider-ek kezelik.

7.7. Többnyelvűség (i18n)

Az Orion platform több nyelvet is támogat. A rendszer a modern i18next megoldást használja a felületi szövegek kezelésére. A fordítások strukturált JSON fájlokban találhatóak a src/locales/ könyvtárban, nyelvi kódok szerint csoportosítva (hu, en, de). A könnyebb karbantarthatóság érdekében a szövegek névterekre (namespaces) vannak bontva: public, user, company, components.

Az oldal betöltésekor a rendszer megnézi, mit választott korábban a felhasználó. Ha még nem választott, automatikusan a magyart (Fallback Language) tölti be.

```

// Példa i18n használatra egy komponensben
const { t } = useTranslation(['public', 'components']);

return (
  <button>
    {t('components:confirm_modal.submit_btn')}
  </button>
);

```

7.8. SEO és Statikus Eszközök

Az Orion platform nem csupán funkcionálisan, hanem technikai keresőoptimalizálás (SEO) szempontjából is modern elvárásoknak felel meg. A robots.txt meghatározza a nyilvános elérést a crawlerek számára és közvetlen linket biztosít a sitemapre. A sitemap.xml tartalmazza az oldal struktúráját és a fontosabb útvonalakat, segítve a Google és más keresők gyorsabb feltérképezését.

A professzionális arculat része a böngésző fülön megjelenő ikon (favicon). Az Orion egy modern, minimalista, csillagképet idéző logót használ, amely nemcsak vizuális azonosítást tesz lehetővé, hanem javítja a Lighthouse "Best Practices" pontszámát is. Minden statikus eszköz a projekt public/ könyvtárában található — a Vite ezeket változtatás nélkül másolja a végleges csomagba (dist).

A Lighthouse tesztek részletes eredményei a [10.4. szakaszban](#) találhatóak.

7.9. Reszponzivitás és adaptív megjelenítés (Responsive Design)

Az Orion frontendje úgy lett kialakítva, hogy a felhasználói élmény minden eszközön (mobil, tablet, desktop) optimális maradjon. A rezponzivitást nem külső CSS keretrendszerrel (mint a Tailwind), hanem egyedi **CSS Modules** alapú megoldásokkal, **Flexbox** és **CSS Grid** elrendezésekkel, valamint finomhangolt **Media Query**-kkel kezeljük.

A rendszer az alábbi főbb töréspontokat (breakpoints) használja az adaptív elrendezéshez:

- **Desktop (1024px felett):** Teljes szélességű elrendezés, háromszlopos Header (Logo - Nav - Actions), komplex dashboard rácsok.
- **Tablet (768px - 1024px):** Szűkített margók, az összetettebb rácsok (pl. ApplicantTrackingSystem) két oszlopra vagy listás nézetre váltanak.
- **Mobil (768px alatt):** Egyoszlopos elrendezés, optimalizált érintési felületek.
- **Kis mobil (480px alatt):** Minimális paddingok, egyszerűsített betűméretek és gombméretek a legkisebb kijelzőkhöz.

Komponens-szintű adaptáció

Bizonyos kritikus komponensek viselkedése jelentősen megváltozik a képernyőméret függvényében:

- **Navigáció (Header):** 768px-es szélesség alatt a központi navigációs linkek (Főoldal, Állások stb.) rejtve maradnak (`display: none`), hogy helyet biztosítsanak a logónak és a funkcionális gomboknak (nyelvválasztó, értesítések, profil).
- **Profil legördülő (ProfileDropdown):** Mobilon a lenyíló menü nem relatív pozicionálású, hanem rögzített (`fixed`), teljes szélességű elemként jelenik meg a képernyő felső részén, megkönnyítve a navigációt.
- **Kártyák és rácsok:** Az álláshirdetések kártyái (JobCard) és a statisztikai panelek dinamikusan méreteződnek, mobilon a vízszintes elrendezés függőlegesre vált.

Ismert korlátozások

Bár az oldal alapvető funkciói mobilon is elérhetőek, jelenleg az alábbi területeken vannak fejlesztés alatt álló elemek:

- A 768px alatt eltűnő főnavigációs linkekhez jelenleg nincs "hamburger menü" implementálva, így ezek az oldalak nehezebben érhetők el mobilon.

8. Külső Integrációk és Speciális Szolgáltatások

TL;DR: A szerveroldali üzleti logika kiegészül két fontos, önállóan működő, speciális rendszerrel: a Nodemailer alapú **Email értesítési és hitelesítési rendszerrel**, valamint a

Python alapú **OrionAI intelligens keresőmotorral**. Ezek nem részei az alap CRUD műveleteknek, hanem külső rendszerekre és külön mikroszolgáltatásokra támaszkodnak.

8.1. Email Integráció és Automatizáció (Nodemailer)

A rendszer az `emailRoute.ts` és `emailService.ts` fájlokon keresztül kezeli az összes platformszintű, automatikus email-kommunikációt. A küldés a **Nodemailer** csomagra épül, egy dedikált Gmail SMTP serveren keresztül (`process.env.EMAIL`). Az email szolgáltatás kritikus fontosságú a biztonságos fiókkezelés szempontjából, mivel ez felel az aszinkron hitelesítési folyamatokért.

Fő folyamatok (Email Services):

- **Fiókaktiválás:** A diákok és a cégek regisztrációja után a rendszer egy 1 óráig érvényes, egyedi jwt alapú tokenet generál (pl. `sendUserActivationEmail`). Ezt a tokenet adatbázisban is rögzíti (`user_email_tokens`), és kiküldi a felhasználónak. A fiók mindaddig inaktív (`activated: false / verified: false`), amíg a linkre rá nem kattintanak.
- **Jelszó visszaállítása:** A `sendUserPasswordResetEmail` és `sendCompanyPasswordResetEmail` funkciók biztonságos, token-alapú visszaállítási linket küldenek. Az új jelszó bekérésekor a szerver (`saveNewUserPassword`) először ellenőrzi a token lejáratát és érvényességét, majd `bcrypt`-tel hasheli és menti az új jelszót, végül törli a felhasznált tokenet az adatbázisból.
- **Tranzakciós rugalmasság:** A szolgáltatás dedikált, időzített token-lejáratú rendszert használ. Ha a felhasználó egy órán belül nem kattint az aktivációs/jelszó linkre, a folyamat biztonsági okokból érvényét veszti.

8.2. OrionAI Integráció (Python NLP)

Az OrionAI (`OrionAI.py` és `OrionAIRoute.ts`) külön útvonalon és külön Python alapú környezetben fut. Míg a rendszer nagy része Node.js alapú, az NLP (Természetes Nyelvfeldolgozás) feladatokat teljesítmény és ökoszisztéma okokból egy önálló Python mikroszolgáltatás látja el. Az Express szerver (az `OrionAIRoute.ts`-en keresztül) csupán API Gateway-ként viselkedik: továbbítja a felhasználói promptokat a Python mikroszolgáltatásnak, és visszajuttatja az eredményt a kliensnek. Ez az elszeparáció biztosítja a magas teljesítményt és azt, hogy az AI számításigényes feladatai ne akasszák meg a fő backend többi folyamatát.

Működési mechanizmus

1. **Input:** A felhasználó természetes nyelven megfogalmazott kérése (pl. "Budapesti diákmunka Python programozóknak").

2. **Paraméterezés:** A rendszer kiegészíti a kérést az elvárt minimális bérrel.
3. **Python ág:** A kérés az Express szerverről a Python alapú OrionAI modulhoz kerül (`OrionAI.py`).
4. **NLP feldolgozás:** A modul elemzi a kérést, kulcsszavakat és szándékot azonosít.
5. **Matchmaking:** Az azonosított feltételek alapján a rendszer rangsorolja a meglévő hirdetéseket.
6. **Eredmény:** A végpont visszaadja a legrelevánsabb hirdetések listáját JSON formátumban.

9. Biztonság, jogosultság és adatvédelem

TL;DR: A JWT és HttpOnly cookie-alapú hitelesítés, valamint a titkosított dokumentumtárolás garantálja a magas szintű adatbiztonságot. A hozzáférést szigorú Route Guard-ok védik, a bemeneti adatokat Zod sémák validálják, a személyes okmányok pedig erős adatbázis-szintű titkosítással (PGP) vannak védve, biztosítva az érzékeny adatok maximális védelmét az egész platformon.

9.1. Auth és session biztonság

Az Orion egyszerre kezel személyes adatokat, céges hozzáféréseket, dokumentumokat és kommunikációs tartalmat, ezért a hitelesítés minősége alapvető. A felhasználók azonosítása modern, kulcs-alapú (JWT – JSON Web Token) megoldással működik. Amikor sikeresen bejelentkezünk, a rendszer kap egy digitális kulcsot. Ennek segítségével maradhatunk bejelentkezve (Session Persistence). Amikor megnyitjuk az oldalt, a rendszer azonnal ellenőrzi ezt a kulcsot. Ez a folyamat észrevétlen, és megakadályozza, hogy a felület ugráljon vagy villogjon (layout shift) betöltés közben.

A JWT architektúra komoly biztonsági és kényelmi előnyöket is nyújt. Egyrészt a tokenek beépített lejáratí idővel (expiration) rendelkeznek, így ha a kulcs illetéktelen kezekbe is kerülne, a lejáratí idő után már nem tudják felhasználni, az automatikusan érvényét veszti. Másrészt a token magában foglalja az alapvető munkamenet-adatokat (például a felhasználó azonosítóját és szerepkörét - diák vagy cég). Így a rendszer biztonságosan megjegyzi a belépett felhasználót, ha az kéri, ezzel elkerülve az állandó, frusztráló újra-bejelentkezéseket. A szerver (a `jsonwebtoken` csomag segítségével) minden egyes kérésnél egy erős titkosítási kulccsal (`JWT_SECRET`) ellenőrzi az aláírást és a lejáratot, biztosítva a maximális adatvédelmet.

9.2. Hibák kezelése és visszajelzés

Egy hálózati rendszernél előfordulhatnak hibák (pl. megszakad a net vagy túlterhelt a szerver). Az Orion 'hibatűrő' (Graceful Degradation) módon működik. Külön tudjuk választani a felhasználói hibákat (pl. rossz jelszó) és a váratlan szerverhibákat, és ezekről mindig egyértelmű üzenetet küldünk (Toast értesítések formájában).

9.3. Szerepköralapú hozzáférés

Mivel a diák és a cég két külön entitás, a hozzáférési szabályok sem lehetnek egyszerűen „bejelentkezett vagy nem bejelentkezett” típusúak. A rendszernek szerepköralapon kell eldöntenie, hogy ki láthat, hozhat létre, vagy módosíthat bizonyos erőforrásokat. Például:

- diák hozhat létre jelentkezést, de nem szerkesztheti más cég hirdetését,
- cég létrehozhat hirdetést és kezelheti saját jelöltjeit, de nem férhet hozzá más cég belső adataihoz,
- chat adatok csak az adott beszélgetés résztvevőinek legyenek hozzáférhetők.

Tehát jogosultságok szempontjából a rendszer egyértelműen szétválasztja a felhasználói és céges részeket, és ezt a felületen is érezteti.

9.4. Titkosított dokumentumkezelés

A rendszer a kiemelt kockázatú személyes adatok (személyazonosító okmányok) védelmét adatbázis szintű titkosítással biztosítja. A tárolás során a PostgreSQL pgcrypto modulja kerül alkalmazásra, amely szimmetrikus titkosítási algoritmusok használatával garantálja a „data-at-rest” (tárolt adatok) védelmét. Ez a megoldás biztosítja, hogy az érzékeny adatok kizárólag titkosított formában kerüljenek mentésre, kiegészítve a hálózati átvitel során alkalmazott védelmi protokollokat.

9.5. Adatminimalizálás és elkülönítés

A hitelesítési adatok a profiladatoktól külön táblákban tárolódnak, ami csökkenti az adatok összekeveredésének kockázatát és egyszerűsíti a jogosultsági logika kezelését. Ez a felosztás lehetővé teszi, hogy a rendszer különböző szintű védelmet alkalmazzon az eltérő érzékenységgű adatokra, biztosítva, hogy a kiemelt biztonságot igénylő információk (például jelszavak vagy azonosítók) elszeparáltan és szigorúbb ellenőrzés mellett legyenek kezelve.

9.6. Frontend oldali biztonsági megfontolások

A frontend oldalon a biztonság elsősorban az autentikációs állapot helyes kezelésében, a jogosultsághoz kötött oldalak védelmében, valamint az érzékeny adatok böngészőoldali tárolásának minimalizálásában jelenik meg.

Az alkalmazás kliensoldali route-védelmet alkalmaz: a felhasználói és vállalati jogosultsághoz kötött oldalak csak megfelelő autentikáció esetén érhetők el, ellenkező esetben a rendszer átirányítja a felhasználót a megfelelő bejelentkezési oldalra. Ez csökkenti annak kockázatát, hogy illetéktelen felhasználó közvetlen URL-hívással védett felületeket jelenítsen meg.

A frontend nem tárol autentikációs tokent `localStorage`-ban vagy `sessionStorage`-ban. Ehelyett a bejelentkezési állapot ellenőrzése a szerverrel történő kommunikáción keresztül valósul meg, sütialapú hitelesítéssel. Ez kedvezőbb megközelítés, mert csökkenti annak kockázatát, hogy egy esetleges kliensoldali kompromittáció során a hitelesítési adatok egyszerűen kiolvashatók legyenek a böngésző tárolójából.

A böngészőoldali tárolás a jelenlegi megvalósításban csak nem érzékeny adatokra korlátozódik, például a kiválasztott nyelvre vagy egyes felhasználói élményt javító ideiglenes állapotokra. Emellett a felület React-alapú renderelést használ, és a vizsgált kódbázisban nem található közvetlen HTML-befecskendezést alkalmazó megoldás, ami hozzájárul az XSS típusú kockázatok mérsékléséhez.

Összességében a frontend oldali biztonsági megoldások ebben a projektben alapvetően a hozzáférés-szabályozásra, a szerveroldali autentikációra való támaszkodásra és az érzékeny adatok kliensoldali kitettségeinek csökkentésére épülnek.

9.7. Kulcsok és környezeti változók

A rendszer működéséhez kritikus biztonsági paramétereket, mint az `ENCRYPTION_KEY` és a `JWT_SECRET`, a backend szigorúan a szerveroldali környezeti változókból (`.env`) olvassa be. A kliensalkalmazás semmilyen formában nem fér hozzá ezekhez a kulcsokhoz. A konfigurációs architektúra révén ezek a titkosítási adatok szigorúan ki vannak zárva a verziókezelésből, ami garantálja, hogy az éles környezetben használt hitelesítési és titkosítási alapok nem kerülhetnek nyilvánosságra és az illetéktelen behatolás esélye a minimálisra csökken.

10. Minősegbiztosítás, tesztelés és teljesítmény

TL;DR: A platform stabilitását és gyorsaságát párhuzamosan kezeljük. A kritikus folyamatokat **háromszintű tesztelési rendszer** (Vitest, RTL, MSW) védi, az átfogó kódstabilitás érdekében. Emellett a teljesítményt folyamatos Lighthouse auditokkal monitorozzuk, és aszinkron lapozással optimalizáljuk az adatkezelést, biztosítva a gyors betöltési időket és a kiváló UX-et.

10.1. Jelenlegi tesztelési eszközök és stratégia

Az Orion frontend projekt tesztelési rendszere azt a célt szolgálja, hogy a felhasználói felület és az üzleti logika módosítások után is megbízhatóan működjön. A jelenlegi tesztelés több szinten valósul meg:

- **Egységtesztek (Unit Tests):** Kisebb, izolált logikai egységek, például validációs sémák tesztelése Vitest segítségével.
- **Komponens tesztek (Component Tests):** Egyes React komponensek működésének és felhasználói interakcióinak ellenőrzése React Testing Library és Vitest használatával.
- **Integrációs tesztek (Integration Tests):** Egyes API-hívásokhoz és adatfolyamatokhoz kapcsolódó működés ellenőrzése MSW segítségével, amely a hálózati kérések szimulálását biztosítja a tesztkörnyezetben.
- **Végponttól végpontig tesztek (End-to-End / E2E Tests):** Alapvető alkalmazás-szintű működés ellenőrzése Playwright használatával, valós böngészőkörnyezethez közeli futtatással.

A projekt csomaglistája és tesztkonfigurációja alapján a főbb eszközök: vitest, @testing-library/react, msw és @playwright/test.

10.2. Manuális API Validáció és Backend Integráció

Annak érdekében, hogy ne csak elméletben (mockolt adatokkal), hanem a gyakorlatban is meggyőződjünk a rendszer stabilitásáról, a fejlesztés során dedikált backend tesztkörnyezetet (Postman) használtunk. Ez a folyamat lehetővé tette, hogy közvetlenül a szerver végpontjait szólítsuk meg, kikerülve a frontend felületét, így izoláltan vizsgálhattuk a biztonsági protokollokat és az adatfolyamokat.

Tesztelt forgatókönyvek és eredmények:

Funkció	Végpont	Tesztelt állapot	Eredmény	Állapotkód
User Login	/auth/user/login	Hibás adatok megadása	"Invalid credentials"	401 Unauthorized
User Login	/auth/user/login	Sikeres bejelentkezés	"Login successful"	200 OK
Company Login	/auth/company/login	Hibás adatok megadása	"Invalid credentials"	401 Unauthorized
Company Login	/auth/company/login	Sikeres bejelentkezés	"Login successful"	200 OK
Adatbiztonság	/advertisements	Lekérés token nélkül	"Missing token"	401 Unauthorized
Adatlekérés	/advertisements	Lekérés érvényes tokennel	Hirdetési lista érkezik	200 OK

Megállapítások:

- **Hitelesítési szétválasztás:** A rendszer megfelelően különválasztja a felhasználói (user) és cégi (company) belépési pontokat, és konzisztens hibaüzeneteket ad vissza.
- **Middleware védelem:** A védett végpontok (pl. /advertisements) megfelelően blokkolják a kéréseket, ha a hitelesítési token (cookie) hiányzik, garantálva az adatok biztonságát.

- **Adatstruktúra:** A sikeres válaszok JSON formátuma (pl. a hirdetések listája) megegyezik a frontend által elvárt sémával, így az integráció zökkenőmentes.

10.3. Teljesítmény és skálázási megfontolások

Az Orion jelenlegi megvalósításában már megjelennek olyan technikai megoldások, amelyek a teljesítmény javítását és a későbbi skálázhatóság támogatását szolgálják, ugyanakkor ezek egyelőre elsősorban alapoptimalizálásoknak tekinthetők.

A frontend oldalon az alkalmazás útvonalai `React.lazy` és `Suspense` használatával kerülnek betöltésre, tehát a késleltetett betöltés jelenleg elsősorban az oldalszintű komponensek szintjén valósul meg. Ez kedvezően hat a kezdeti betöltésre, mert nem szükséges az összes oldal kódját az alkalmazás indulásakor egyszerre letölteni. Ugyanakkor fontos kiemelni, hogy ez nem jelent teljes körű, komponensszintű lazy loading stratégiát, hanem elsősorban az útvonalak szerinti felosztást valósítja meg.

A hirdetések listázása lapozott módon történik. A kliensoldal egyszerre csak egy meghatározott számú elemet kér le, majd szükség esetén további tételeket tölt be. Ezt a backendoldali lekérdezés is támogatja `page` és `limit` paraméterek segítségével, így a rendszer nem egyszerre próbálja megjeleníteni az összes rekordot. Ez a megközelítés különösen fontos lehet nagyobb adathalmazok esetén.

A keresési és szűrési funkciók `debounce` mechanizmust alkalmaznak, amely csökkenti a gépelés közben elindított felesleges hálózati kérések számát. Ez mérsékli mind a kliensoldali, mind a szerveroldali terhelést, és egyenletesebb felhasználói élményt biztosít.

A chatfunkció valós idejű kommunikációra épül `WebSocket` kapcsolat használatával, automatikus újracsatlakozási logikával és `optimista` felületfrissítéssel. Ez javítja a rendszer válaszkészségét, ugyanakkor a felhasználók és az egyidejű kapcsolatok számának növekedésével a kapcsolatok kezelése és az állapotfrissítések száma skálázási szempontból külön figyelmet igényelhet. Az értesítési rendszer ezzel szemben jelenleg időszakos lekérdezéssel működik, ami egyszerű és megbízható megoldás, de nagyobb terhelés mellett kevésbé hatékony, mint egy valós idejű push-alapú mechanizmus.

Összességében az Orion jelenlegi architektúrája már tartalmaz néhány fontos teljesítményjavító elemet, például az útvonalalapú lazy loadingot, a lapozott adatlekérést és a `debounce` alapú keresést, ugyanakkor a további optimalizációk között hosszabb távon indokolt lehet a finomabb kódfelosztás, a listanézetek további optimalizálása, valamint az értesítési rendszer valós idejűbb működésének kialakítása.

10.4. Lighthouse Audit és mérőszámok

WEB-AUDIT ÖSSZESÍTÉS

Az Orion platform Lighthouse auditja összességében kedvező eredményeket mutat az elérhetőség, a bevált webes gyakorlatok és a keresőoptimalizálás terén, miközben a teljesítménymutatók alapján további optimalizációs lehetőségek is azonosíthatók.

67 PERFORMANCE	92 ACCESSIBILITY	100 BEST PRACTICES	100 SEO
--------------------------	----------------------------	------------------------------	-------------------

Elemzés és következtetések:

- **SEO és Best Practices (100%):** A platform ezen a két területen kiváló eredményt ért el, ami arra utal, hogy az alkalmazás megfelel az általános technikai és keresőoptimalizálási elvárásoknak.
- **Accessibility (92%):** Az akadálymentesítés szintje magas, ugyanakkor néhány kisebb finomhangolás, például bizonyos ARIA-attribútumok pontosítása, tovább javíthatja az eredményt.
- **Performance (67%):** A teljesítménymutató alapján az alkalmazás működése már elfogadható, de további optimalizálás indokolt. A projekt jelenleg útvonalalapú lazy loadingot alkalmaz, azonban ez még nem jelent teljes körű, finomabb szintű kódfeosztást. A teljesítményt várhatóan elsősorban a betöltődő JavaScript mennyisége, valamint az első megjelenítéshez kapcsolódó feldolgozási költség befolyásolja.

Kulcsfontosságú mérőszámok:

- **FCP (First Contentful Paint):** 1.3 s
- **LCP (Largest Contentful Paint):** 5.5 s
- **Speed Index:** 1.9 s
- **TBT (Total Blocking Time):** 0 ms
- **CLS (Cumulative Layout Shift):** 0.058

11. Fejlesztői működés, build és üzemeltetés

11.1. Helyi fejlesztői környezet

Az Orion projektet kétféleképpen indíthatod el helyben: futtathatod közvetlenül a forráskódból a fejlesztői környezet beállításával, vagy használhatod az automatizált telepítőcsomagot. Míg az előbbi a fejlesztőknek nyújt nagyobb szabadságot, az utóbbi gyorsabb és egyszerűbb üzembe helyezést biztosít.

A manuális telepítéshez klónozd a projektet a `git clone` paranccsal. Ebben az esetben neked kell előkészítened a futtatókörnyezetet: telepítened kell a Node.js függőségeket a frontendhez és a backendhez, valamint be kell állítanod a Python környezetet az OrionAI modulhoz. A rendszer felépítéséből adódóan a frontend egy Vite szerveren, a backend egy Node.js folyamatként, az AI modul pedig egy önálló Python folyamatként fut.

A GitHub release-ek között találsz egy előre konfigurált telepítőt (Orion-Setup.exe) is. Ez a csomag a `start.bat` fájl futtatásával automatikusan elvégzi a munka nehezét: ellenőrzi a szoftveres feltételeket, szükség esetén telepíti a Node.js-t és a Pythont, létrehozza a virtuális környezetet (venv), és letölti a függőségeket. Végül egyszerre indítja el a backendet, a frontendet és az AI kiszolgálót, így az első indítás minimális kézi beavatkozást igényel.

Fontos: Bármelyik utat is választod, A backend server indításához elengedhetetlen a megfelelő `.env` konfiguráció, benne az adatbázis-kapcsolati és a titkosításhoz szükséges egyedi kulcsokkal.

```
git clone [repository-url]
cd Orion
npm install
npm run dev           # Frontend indítása (Vite)
npm run server        # Express backend indítása
npm run OrionAI       # AI modul indítása
```

11.2. Környezeti változók

A backend indításához szükséges a megfelelő `.env` konfiguráció. A szerverfájl pedig név szerint hivatkozik bizonyos kulcsokra. Ez alapján minimálisan elvárt, hogy a projekt rendelkezzen a következő változókkal:

- SUPABASE_SERVICE_KEY
- ENCRYPTION_KEY

- JWT_SECRET
- EMAIL
- EMAIL_PASSWORD

11.3. Build és release szemlélet

A build folyamat a TypeScript fordítást és a Vite buildet kombinálja. Ez azt jelzi, hogy a kliensoldali csomagolás és a típuskövetkezetesség együtt alkotják a release előszűrését. A backend külön Node-alapú futtatást követ, ezért az üzembe helyezés során célszerű a két réteget külön release egységként kezelni, még akkor is, ha ugyanabban a repositoryban találhatók.

12. Jövőbeli fejlesztési irányok és tervek

Az Orion platform stabil alapokon nyugszik, ami lehetővé teszi további üzleti és felhasználói funkciók integrálását. A jövőbeni tervek elsősorban az ökoszisztéma bővítésére, a bevételi források megteremtésére és a felhasználói élmény növelésére fókuszálnak.

12.1. Üzleti és monetizációs tervek

- **Fizetett reklámok és kiemelések bevezetése:** A munkáltatók (cégek) számára elérhetővé válik a hirdetések kiemelése extra díj ellenében. Ezek a hirdetések a keresési találatok és a főoldal tetején jelennének meg ("Szponzorált" vagy "Kiemelt" címkével), jelentősen növelve a láthatóságot.
- **Prémium céges profilok:** Előfizetéses modell bevezetése cégek számára, amely részletesebb analitikát, dedikált arculati elemeket (pl. egyedi profil design) és automatikus előszűrési opciókat biztosít a jelentkezők kezelésénél.

12.2. Felhasználói élmény és kényelmi funkciók

- **Naptár integráció (Google Calendar / Apple Calendar):** A sikeresen elfogadott jelentkezések, megbeszéltek interjúk és a ledolgozandó műszakok automatikusan exportálhatók lesznek a diákok saját naptárába, megkönnyítve az időbeosztást.
- **Mobilalkalmazás fejlesztése:** Bár a webes felület reszponzív, a valós idejű push-értesítések és a gyorsabb chat-kommunikáció érdekében egy dedikált (pl. React Native alapú) mobilalkalmazás kiadása is szerepel a hosszú távú tervek között.
- **Kibővített OrionAI funkciók:** A mesterséges intelligencia a jövőben nemcsak a hirdetések keresésében fog segíteni, hanem a diákok számára proaktívan felvázolhatja, milyen extra képességekre lenne szükségük a vágyott pozíciók betöltéséhez, illetve segíthet a CV tartalmának optimalizálásában.

12.3. Technikai és infrastrukturális továbbfejlesztések

- **Adatbázis skálázás és archiválás:** A lejárt és inaktív hirdetések, valamint a régi jelentkezések automatikus hideg-tárhelyre mozgatása (Cold Storage) az aktív adatbázis teljesítményének megőrzése érdekében.

- **Automatizált tesztelési lefedettség bővítése:** A fizetési és hirdetés-kiemelési folyamatok bevezetésével párhuzamosan a kritikus üzleti folyamatok teljes körű E2E tesztelése elengedhetetlenné válik.
- **API verziókezelés (v1, v2):** Az új funkciók és a potenciális mobilalkalmazás bevezetésével a backend végpontjait szigorú verziókezeléssel kell ellátni, garantálva a régebbi kliensek visszafelé kompatibilitását.

13. Fejlesztési folyamat és ütemezés

TL;DR: Az Orion fejlesztését **két fős csapatunk** végezte agilis szemlélettel, **Trello** alapú feladatkezelést és **Discord** kommunikációt alkalmazva. A fejlesztés során kiemelt figyelmet fordítottunk a **Clean Code** alapelvekre és a fenntartható architektúrára. A projektet a tervezéstől a tesztelésig hat fő fázisra bontottuk.

13.1. Csapat és szerepkörök

A projekten ketten dolgoztunk, a felelősségi köröket az alábbiak szerint osztottuk el:

Fejlesztő	Felelősségi kör	Főbb feladatok
Peti	Backend fejlesztés és adatbázis	Express API végpontok, szolgáltatási réteg, Supabase adatbázis-tervezés (táblák, RPC-k, triggerek), JWT hitelesítés, WebSocket integráció, OrionAI összekötés
Domonkos	Frontend fejlesztés	React komponensek, oldalak és nézetek, routing és Route Guard rendszer, Vanilla CSS design, Framer Motion animációk, űrlapkezelés (React Hook Form + Zod)

13.2. Eszközök és kommunikáció

- Feladatkezelés – Trello:** A feladatok nyomon követésére Trello táblát használtunk, ahol a kártyákat a szokásos Kanban-oszlopok mentén mozgattuk (Teendő → Folyamatban → Kész). Minden nagyobb funkciót és hibajavítást külön kártyaként kezeltünk, így a haladásunk bármikor átlátható volt.
- Kommunikáció – Discord:** A napi egyeztetéseinket, technikai döntéseinket és a kódintegrációs kérdések tisztázását saját Discord szerverünkön zajlott. A szöveges és hangcsatornák lehetővé tették az azonnali problémamegoldást és a párhuzamos fejlesztési ágaink összehangolását.

- **Verziókezelés – Git / GitHub:** A kódbázist a DomonkosTech/Orion GitHub repositoryban tároltuk, ahol külön ágakon dolgoztunk és pull request-ekkel integráltuk a változtatásokat.

13.3. Fejlesztési ütemterv

A projekt fejlesztését az alábbi fő fázisokra bontottuk. Az ütemezést rugalmasan, iteratív módon kezeltük, az egyes fázisok között tudatosan hagytunk átfedéseket.

Fázis	Időszak (hét)	Fő tevékenységek
1. Tervezés és architektúra	1–2. hét	Követelmények összegyűjtése, adatmodell megtervezése, technológiai verem kiválasztása, Trello tábla felállítása, fejlesztési környezet konfigurálása.
2. Alaprendszer kiépítése	3–9. hét	Adatbázis-séma létrehozása (táblák, FK kapcsolatok), Express szerver alap middleware-ek, JWT hitelesítés, frontend vázszerkezet (routing, layout komponensek).
3. Fő funkciók implementálása	9–17. hét	Regisztráció és bejelentkezés (diák + cég), profilkezelés, álláshirdetés CRUD műveletek, jelentkezési rendszer (ATS) alapjai, céges dashboard.
4. Kommunikáció és értesítések	17–21. hét	OrionMessenger (valós idejű chat) implementálása, WebSocket integráció, rendszerüzenetek, email-értesítési rendszer (Nodemailer) bekötése.
5. Intelligens funkciók és finomhangolás	21–26. hét	OrionAI NLP keresőmotor integrálása, kedvencek rendszere, statisztikák, UI animációk és reszponzív finomhangolás.

Fázis	Időszak (hét)	Fő tevékenységek
6. Tesztelés és dokumentáció	26–28. hét	Manuális végponttesztelés, biztonsági ellenőrzések, Lighthouse teljesítménymérés, hibajavítások, rendszerdokumentáció véglegesítése.

13.4. Fejlesztési folyamat összefoglalása

A fejlesztés során az alábbi ciklikus munkafolyamatot követtük:

1. **Feladatkijelölés:** A Trello táblán kiválasztott kártyát „Folyamatban” státuszba helyeztük.
2. **Implementáció:** A backend és frontend fejlesztéssel párhuzamosan haladtunk; a közös API interfészeket előre egyeztettük, hogy az integráció során minimális legyen a súrlódás.
3. **Integráció:** A kész funkciókat GitHub-on egyesítettük és helyi környezetben teszteltük az összjátékot.
4. **Ellenőrzés:** Manuális tesztek és kódellenőrzés után a feladatot „Kész” státuszba tettük.
5. **Iteráció:** A visszajelzések alapján szükség esetén elvégeztük a hibajavításokat és a finomhangolást.

14. Mellékletek

Hamarosan...

Orion teljes rendszerdokumentáció.